

Premessa

Il 14 febbraio 2012, al Massachusetts Institute of Technology, Jeff Bezanson, Stefan Karpinski, Viral Shah e Alan Edelman hanno reso disponibile un nuovo linguaggio di programmazione, da loro definito utile per il calcolo scientifico, il machine learning, il data mining, l'algebra lineare, il calcolo parallelo e il calcolo distribuito.

Si coronava così il sogno dei quattro volonterosi che da tre anni si erano posti il dichiarato obiettivo di superare i compromessi tra Matlab, Lisp, Python, Ruby, Perl, Mathematica, R e C, radunando in un solo linguaggio quanto di meglio è presente in quei linguaggi per il calcolo.

Il nome Julia è stato scelto senza particolari motivi: semplicemente perché da Karpinski è stato considerato un bel nome e un amico di Bezanson l'aveva suggerito.

Julia presenta i vantaggi di essere un linguaggio relativamente facile da apprendere e da usare e di eseguire calcoli, anche complicati e coinvolgenti masse enormi di dati, con una velocità non riscontrabile altrove nei così detti linguaggi interpretati, come Java, Python, Lisp, ecc., e pari a quella dei linguaggi compilati, come il C, tuttavia molto più difficile da apprendere e da utilizzare. Uno degli slogan di cui ama fregiarsi Julia è «Walks like Python. Runs like C».

La eccezionale velocità di elaborazione di questo linguaggio deriva dal fatto che Julia non lavora, come avviene negli altri casi di linguaggi interpretati, eseguendo le istruzioni dello script una per volta ma compila tutto lo script in linguaggio macchina e poi lo esegue. Da qui un riscontrabile piccolo ritardo nella partenza dell'esecuzione con un vistoso recupero nell'esecuzione una volta che è partita: ciò rende poco percepibile la velocità di Julia nell'esecuzione di script semplici, come quelli che affronteremo in questo manuale, ma la velocità di Julia si manifesta nell'esecuzione di script impegnativi specialmente quando si è in presenza di grandi masse di dati da elaborare.

Queste caratteristiche fanno di Julia un linguaggio che promette di insidiare la primazia di Python per la data science e il big data mining.

Dal momento del rilascio Julia è diventato open source e ciò ha contribuito agli arricchimenti del linguaggio che si sono accumulati in pochi anni ma i suoi progettisti originari, con altri due soci, nel luglio del 2015 hanno fondato la società Julia Computing per "sviluppare prodotti che rendano Julia facile da usare, facile da implementare e facile da ridimensionare". Oltre a mantenere i repository open source Julia su GitHub, Julia Computing offre prodotti commerciali, incluso JuliaPro, disponibile sia in versione gratuita senza assistenza sia in versione a pagamento con assistenza professionale di primissimo livello.

In questo manuale mi propongo di illustrare il funzionamento di base di Julia, conducendo il lettore alla realizzazione di script elementari da dilettanti.

Navigando in rete si possono trovare ottimi testi in lingua inglese per approfondimenti da professionisti, primo fra tutti il manuale ufficiale che troviamo, in formato PDF, all'indirizzo <https://julialang.org/>, nella pagina DOCUMENTATION.

Unico testo con un minimo di organicità su Julia in lingua italiana penso sia quello rintracciabile su <https://riptutorial.com/it/julia-lang> nel file `julia-language-it.pdf`, ma non è più aggiornato dalla superata versione di julia 0.5 del settembre 2016.

Indice

1	Il mondo di Julia	4
1.1	Il pacchetto base e la sua installazione	4
1.2	Il repository di Julia	4
2	Basi del linguaggio	4
2.1	Come funziona	4
2.2	Tipi	6
2.2.1	Tipi base	6
2.2.2	Tipi contenitori	7
2.3	Variabili	8
2.4	Operatori	9
2.4.1	Operatori aritmetici	9
2.4.2	Operatori di confronto	10
2.4.3	Operatori logici	10
2.5	Interattività con l'utente	10
2.5.1	Output	10
2.5.2	Input	10
2.6	Strutture di controllo	11
2.6.1	Esecuzione condizionale	11
2.6.2	Ripetizione	12
2.7	Semplici programmi	13
2.8	Funzioni	16
2.9	Funzioni preconfezionate di base	17
2.9.1	Funzioni aritmetiche	17
2.9.2	Funzioni trigonometriche	17
2.9.3	Funzioni relative ad array numerici	17
2.10	Dot syntax e Funzioni anonime	18
2.11	Librerie standard	18
2.11.1	Printf	19
2.11.2	LinearAlgebra	19
2.11.3	Statistics	19
2.12	Orientamento agli oggetti	20
2.13	Lavorare con file	20
2.14	Packages	21
2.15	Esercizi conclusivi	22
3	Grafica per GUI	24
4	Grafica per la rappresentazione di dati	25
4.1	Plots	25
4.1.1	plot()	26
4.1.2	scatter()	28
4.1.3	bar()	29
4.1.4	pie()	30
4.2	Makie	30
4.3	PyPlot	30
4.3.1	plot()	31
4.3.2	bar()	33
4.3.3	pie()	34

5	Julia per la scienza e il deep learning	35
5.1	Calcolo simbolico con Symbolics.jl	35
5.1.1	Operazioni di base	36
5.1.2	Elaborazione delle espressioni	38
5.1.3	Calcolo	39
5.1.4	Risolutori	39
5.1.5	Matrici	39
5.2	Statistica inferenziale	40
5.3	Matematica superiore	40
5.4	Machine learning	40
5.5	Manipolazione dati	41
5.6	Deep learning	41

1 Il mondo di Julia

1.1 Il pacchetto base e la sua installazione

Julia si trova all'indirizzo <https://julialang.org/>.

Cliccando sul pulsante INSTALL VX.XX.X apriamo la pagina per l'installazione.

Aperto la pagina DOCS abbiamo un ricco manuale in lingua inglese, consultabile online o scaricabile in formato PDF.

La pagina per l'installazione si apre riferendosi al sistema operativo che stiamo utilizzando e ci indica il comando da utilizzare a terminale per installare l'ultima versione di Julia e il tool `juliaup`.

Se siamo su Windows possiamo installare il tutto ricorrendo a Microsoft Store.

Il tool `juliaup` ci consente di gestire installazioni, disinstallazioni e aggiornamenti di versioni di Julia e si utilizza scrivendo a terminale il comando `juliaup` seguito dal nome dell'azione che vogliamo eseguire. I comandi principali sono i seguenti:

`juliaup status` mostra le versioni di Julia installate, indicando quella di default,

`juliaup update` installa l'ultima versione disponibile di Julia sostituendo la vecchia,

`juliaup add x.x.x` installa una versione specifica di Julia aggiungendola alle altre,

`juliaup remove x.x.x` disinstalla una versione specifica di Julia,

`juliaup self update` aggiorna la versione di `juliaup`.

1.2 Il repository di Julia

Nel momento in cui scrivo, all'indirizzo

<https://juliapackages.com/>

troviamo catalogati oltre 12.000 pacchetti con cui fare cose che esulano dalle tantissime che possiamo fare semplicemente affidandoci al pacchetto base.

Possiamo consultare il catalogo avvalendoci della utile suddivisione per categorie che ci viene proposta.

Cliccando sul nome del pacchetto apriamo una pagina dove ne troviamo una sommaria descrizione con eventuali rimandi a documentazione specifica.

Informazioni aggiuntive possiamo avere cliccando, in questa pagina descrittiva, sui pulsanti WEBSITE e GITHUB.

L'installazione e la disinstallazione dei pacchetti avviene in modo molto semplice, come vedremo nel seguito.

* * *

A proposito di pacchetti aggiuntivi è bene sapere che Julia può utilizzare, nei modi che vedremo, funzioni del linguaggio Python attraverso il package **PyCall**, importando il modulo Python che le contiene.

Operando in questo modo rinunciamo ovviamente al pregio della velocità di elaborazione di Julia, raggiungibile soltanto operando con pacchetti aggiuntivi del proprio repository.

2 Basi del linguaggio

2.1 Come funziona

Julia è un linguaggio interpretato e interattivo.

Interpretato significa che il codice che scriviamo viene eseguito direttamente da uno script.

Interattivo significa che possiamo anche vedere eseguite le nostre istruzioni intanto che le scriviamo.

Quello che abbiamo installato secondo quanto indicato nel precedente Capitolo è l'interprete e viene avviato con il comando a terminale `julia`.

Se ci proponiamo di realizzare il nostro primo programma Ciao mondo possiamo sperimentarlo in due modi.

Possiamo scrivere in un file di testo, con un qualsiasi editor di testo, questa istruzione `println("Ciao mondo")`

e salvare il file con estensione `.jl`: `ciaomondo.jl`.

Successivamente, e tutte le volte che vogliamo, posizionandoci nella directory dove abbiamo salvato il file e scrivendo a terminale

```
julia ciaomondo.jl
```

vedremo eseguito il nostro programma (meglio chiamato script) che produrrà la voluta scritta.

L'altro modo è quello, interattivo, di avviare l'interprete scrivendo a terminale il comando

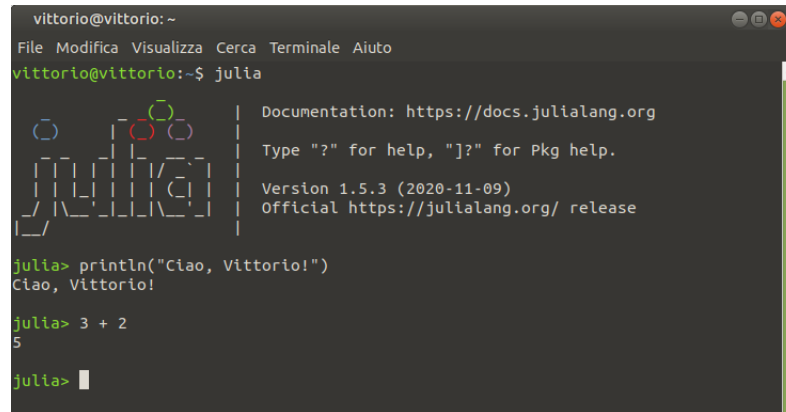
```
julia
```

col che il terminale si trasforma in terminale Julia, la shell Julia (chiamata anche REPL, per Read-Evaluate-Print-Loop), una conchiglia all'interno della quale si usa il linguaggio Julia, e vi possiamo scrivere l'istruzione

```
println("Ciao mondo")
```

vedendola immediatamente eseguita alla pressione del tasto INVIO.

La shell di Julia, nel mio terminale Linux, si presenta così



```
vittorio@vittorio: ~
File Modifica Visualizza Cerca Terminale Aiuto
vittorio@vittorio:~$ julia

Documentation: https://docs.julialang.org
Type "?" for help, "]?" for Pkg help.
Version 1.5.3 (2020-11-09)
Official https://julialang.org/ release

julia> println("Ciao, Vittorio!")
Ciao, Vittorio!

julia> 3 + 2
5

julia> 
```

La modalità interattiva è molto utile per vedere in diretta l'effetto delle nostre istruzioni. Nell'illustrazione vediamo a terminale l'istruzione `julia` che apre la shell e mostra il prompt `julia>`.

Scriviamo in corrispondenza al prompt un'istruzione in linguaggio Julia e premendo Invio ne vediamo immediatamente l'effetto: sono esemplificate una istruzione per un saluto e una istruzione per sommare due numeri.

Quando l'effetto dell'istruzione è un risultato numerico, l'ultimo risultato ottenuto viene memorizzato in una variabile chiamata `ans` (che sta per answer, risposta). Nel caso dell'illustrazione, se, al punto in cui siamo, scriviamo l'istruzione `ans * 6` otteniamo il risultato 30.

Se non ci interessa vedere il risultato dell'istruzione, la terminiamo con un punto e virgola (`;`). In tal modo non vedremo il risultato, che verrà tuttavia memorizzato nella variabile `ans`. Così, se scriviamo `3 * 2;` non vedremo il risultato 6 premendo INVIO, ma, se immediatamente dopo scriviamo `ans + 2`, vedremo il risultato 8.

La shell di Julia può anche essere utilizzata per accedere al volo alla descrizione di una istruzione del linguaggio.

Per fare questo, in corrispondenza del prompt `julia>` scriviamo `?` e il prompt si trasforma nel prompt `help?>`. Se, in corrispondenza di questo nuovo prompt scriviamo un'istruzione del linguaggio, ad esempio `println`, ne otterremo la descrizione ed esemplificazioni sul suo utilizzo.

La shell di Julia può anche momentaneamente tornare ad essere terminale di sistema digitando il carattere punto e virgola in corrispondenza al suo prompt.

Se premiamo INVIO nella shell di Julia prima di aver completato un'istruzione che Julia riconosca possiamo completare l'istruzione nella riga successiva e solo dopo che l'istruzione sarà completa e riconosciuta, alla pressione di INVIO, verrà eseguita.

Se vogliamo far eseguire a Julia, attraverso la shell, più istruzioni insieme dobbiamo elencarle tra la parola chiave `begin` e la parola chiave `end` oppure scriverle, separate da punto e virgola, tra parentesi tonde.

Ad esempio, se scriviamo al prompt

```
begin
print("3 volte 2 fa ")
3*2
end
```

alla pressione di INVIO dopo aver scritto `end` abbiamo il risultato

```
3 volte 2 fa 6.
```

Come se avessimo scritto `(print("3 volte 2 fa "); 3 * 2)`.

Possiamo scoprire altre finchezze sulla shell Julia andando alla voce `Standard Library` del manuale di Julia alla sottovoce `The Julia REPL`.

2.2 Tipi

Julia è un linguaggio a tipizzazione dinamica e non richiede che il tipo di dato da elaborare sia preventivamente indicato dall'utente: a seconda della sintassi con cui è espresso il dato da elaborare Julia assegna al dato stesso il tipo.

I tipi base di Julia sono i soliti che troviamo nei linguaggi di programmazione (numeri, caratteri, stringhe).

2.2.1 Tipi base

Numeri

Julia tratta numeri interi, numeri in virgola mobile (dotati del separatore decimale `.`) e numeri complessi.

Per i numeri interi (`Int`) esiste un'ampia scelta ricollegabile alla dimensione: si va da `Int8` (intero con segno a 8 bit compreso -2^7 e $2^7 - 1$, cioè tra -128 a 127) a `Int128` (intero con segno a 128 bit compreso tra 0 e $2^{127} - 1$, che è un bel numero a 39 cifre). Nei sistemi operativi a 64 bit Julia assegna automaticamente al dato numerico intero (senza il punto decimale) il tipo `Int64` (intero di 19 cifre).

Per i numeri in virgola mobile (`Float`), quelli indicati con il punto separatore decimale, abbiamo due tipi: `Float32` per la precisione singola (8 cifre significative) e `Float64` per la precisione doppia (16 cifre significative). Nei sistemi operativi a 64 bit Julia assegna automaticamente al dato numerico con punto di separazione decimale il tipo `Float64`.

Tutte queste limitazioni dimensionali sono finalizzate alla velocità di elaborazione. Se dobbiamo lavorare con grandi numeri possiamo rinunciare alla velocità e lavorare in precisione arbitraria ottenendo numeri grandi quanto ce lo consente l'hardware su cui lavoriamo (come avviene normalmente se usiamo Python). A questo scopo basta che un solo operando venga scritto come `BigInt(<numero>)` o come `BigFloat(<numero>)`, anche semplicemente `big(<numero>)`.

Nei numeri complessi (`Complex`) parte reale e parte immaginaria, dotate di segno, sono scritte di seguito e la parte immaginaria è contrassegnata dal suffisso `im`.

Esempi:

```
728 è un numero intero,
7.5 è un numero in virgola mobile,
-1+0.78im è un numero complesso.
```

Caratteri

Il carattere (`Char`) è un carattere racchiuso tra apici semplici (`'`).

Stringhe

La stringa (String) è una sequenza di caratteri racchiusa tra apici doppi (").

La stringa è immutabile, cioè non è possibile aggiungervi altri caratteri o toglierne.

La posizione del carattere è indicizzata partendo da 1 (non da 0 come avviene quasi sempre in informatica). La lunghezza della stringa viene indicata dalla funzione `length(<stringa>)`.

Esempi:

"Vittorio" è una stringa,

"Vittorio"[5] è il quinto carattere della stringa "Vittorio", cioè o.

Valori booleani

I valori booleani (Bool) sono true (vero) e false (falso).

2.2.2 Tipi contenitori

Sono insiemi variamente organizzati di elementi di un tipo base.

L'elemento può essere indicato letteralmente o attraverso il richiamo di una variabile che lo contiene. Il suo numero viene ottenuto con la funzione `length()`.

Tupla

La tupla (Tuple) è una sequenza di elementi eterogenei, racchiusi tra parentesi tonde e separati da una virgola (,).

La tupla è immutabile, cioè non è possibile aggiungervi altri elementi o toglierne.

La posizione dei suoi elementi è indicizzata partendo da 1.

Esempio:

(1, 24, "giuseppe", 'p', A) è una tupla, composta da due interi, una stringa, un carattere e dal contenuto della esistente variabile A.

Dizionario

Il dizionario (Dict) è una sequenza di elementi eterogenei contraddistinti da una chiave accoppiata a ciascun elemento.

Viene creato inserendo chiavi e elementi tra due parentesi tonde precedute dalla parola chiave Dict.

Chiave e relativo elemento sono separati dal simbolo => e gli accoppiamenti sono separati da virgola (,).

Esempio:

Dict('a' => 35, 3 => "pippo", 5 => 78) è un dizionario e 'a', 3 e 5 sono le chiavi.

Dict('a' => 35, 3 => "pippo", 5 => 78)[3] individua l'elemento corrispondente alla chiave 3, cioè la stringa "pippo".

Array

L'array (Array) è una collezione ordinata di elementi racchiusi tra parentesi quadre ([e]) e disponibili su più righe e colonne. Gli elementi di ogni riga si indicano separati da uno spazio e inserendo un punto e virgola si passa a una riga successiva. Il numero di elementi per ogni riga, che corrisponde al numero di colonne, deve sempre essere uguale.

Un array può essere composto da una sola riga (indicando gli elementi separati da spazio): in tal caso si parla anche di matrice monodimensionale; oppure può essere composto di una sola colonna (indicando gli elementi separati dal punto e virgola): in tal caso si parla di vettore.

L'array con più righe e più colonne è una matrice pluridimensionale.

La posizione degli elementi è indicizzata partendo da 1. Per gli array su una sola riga o su una sola colonna l'indice indica la posizione nella riga, partendo da sinistra, o nella colonna, partendo dall'alto. Per gli array su più righe la posizione è individuata da un indicatore composto dall'indice di riga e, separato da una virgola, dall'indice di colonna.

Esempi:

```
['a' 2 "pippo"] è un array su una sola riga,  
['a' 2 "pippo"][3] individua il suo terzo elemento "pippo",  
[5; "Luigi"; true] è un array su una sola colonna,  
[5; "Luigi"; true][2] individua il suo secondo elemento "Luigi",  
[1 'g' 12; "pippo" 24 'a'] è un array di due righe e tre colonne,  
[1 'g' 12; "pippo" 24 'a'] [2,3] individua il terzo elemento sulla seconda riga 'a'.
```

Gli array esclusivamente composti di elementi numerici sono di fondamentale importanza per l'algebra lineare.

Per chi conosce il linguaggio Python, che non contempla l'array se non attraverso il modulo aggiuntivo Numpy e, nella versione di base, contempla il tipo List, sottolineo come, in Julia, dove invece abbiamo l'array ma non la lista, l'array a una dimensione possa essere utilizzato come lista, cioè un tipo contenitore di un numero variabile di elementi eterogenei.

Un array può essere inizializzato con il comando `[ ]` e gli elementi possono essere inseriti con `push!(<array>, <elemento>)` per creare un array colonna (vettore).

Inserendo più vettori colonna separati da spazio tra `[ e ]` costruiamo una matrice. In uno script occorre preventivamente inizializzare la matrice con `[ ]` e inserire la prima colonna con la funzione `vcats [<matrice> <colonna>]`.

* * *

Esiste la funzione `typeof()`, attraverso la quale possiamo verificare quale tipo Julia assegna alla cosa che scriviamo.

Per esempio, se scriviamo nella shell di Julia `typeof(122)`, viene ritornato `Int64`, a dimostrazione del fatto che Julia ha capito che abbiamo scritto un numero intero e automaticamente lo tratta a 64 bit.

2.3 Variabili

Le variabili sono delle locazioni di memoria, delle scatole, alle quali diamo un nome, destinate a contenere valori di un certo tipo.

In Julia le variabili si creano nel momento in cui servono, senza bisogno, come avviene in molti linguaggi di programmazione, di dichiararle prima.

La tipizzazione della variabile avviene in maniera dinamica automaticamente al momento dell'assegnazione del valore, in quanto Julia, come abbiamo visto nel precedente Paragrafo, riconosce il tipo da come scriviamo il valore stesso.

L'assegnazione del valore viene fatta con l'operatore `=` e la sintassi

```
<nome_variabile> = <valore>.
```

```
nome = "Vittorio" crea la variabile nome e le assegna il valore di tipo String Vittorio
```

```
raggio = 6.5 crea la variabile raggio e le assegna il valore di tipo Float64 6,5
```

```
A = [15 22 8.5] crea l'array (vettore riga) A assegnandogli i valori 15 22 8.5
```

```
l = [] crea la variabile l contenente un array/lista vuoto
```

```
push!(l, 15) inserisce nell'array/lista contenuto nella variabile l l'elemento numerico 15.
```

Il valore può essere espresso in modo letterale, come negli esempi, o in forma di espressione matematica, più o meno coinvolgendo altre variabili.

Può accadere di dover modificare il tipo di una variabile, per esempio per poter applicare ad essa metodi che non sarebbero propri del tipo assegnato dinamicamente al momento della sua creazione.

I casi più frequenti sono quelli di dover convertire in stringa un valore numerico e viceversa.

Per convertire un numero in stringa usiamo la funzione

```
string()
```

passando come argomento tra le parentesi il numero da convertire o il nome della variabile che lo contiene.

Per convertire una stringa in numero usiamo la funzione `parse(<tipo>, <cosa>)`

dove `<tipo>` è il tipo numerico che scegliamo: se indichiamo `Int` Julia intende `Int64` oppure indichiamo noi il tipo voluto; per il tipo `Float` dobbiamo indicare noi `Float32` o `Float64`; `<cosa>` è la stringa da convertire o il nome della variabile che la contiene.

Le variabili create nello script sono variabili globali. Le variabili create in un blocco di istruzioni all'interno dello script (ne riparleremo quando parleremo di cicli di controllo e di funzioni) sono variabili locali. In un blocco di istruzioni interno allo script si usano normalmente variabili locali; se vogliamo utilizzare una variabile globale dobbiamo richiamarla con il suo nome preceduto dalla parola chiave `global`.

Per le variabili globali non è possibile indicare il tipo di valore che sono destinate a contenere. Esso viene esclusivamente determinato dalla natura del dato inserito.

Per la variabili locali possiamo determinare il tipo nei modi che vedremo parlando delle funzioni.

2.4 Operatori

Gli operatori collegano tra loro operandi in espressioni che forniscono un risultato.

2.4.1 Operatori aritmetici

In ordine di precedenza di esecuzione sono i seguenti:

- `^` per l'elevamento a potenza,
- `//` per la creazione di una frazione,
- `*` per la moltiplicazione,
- `/` per la divisione,
- `%` per il modulo (resto della divisione intera),
- `+` per la somma,
- `-` per la sottrazione.

L'operatore per la creazione di frazioni penso sia esclusivo del linguaggio Julia. Inserito tra numeratore e denominatore crea una frazione che si può assoggettare a calcolo, come fosse un numero, e il risultato viene espresso a sua volta come frazione.

Esempi:

```
3//2 + 1 dà 5//2
3//2 + 1//2 dà 2//1
(5//4)^2 dà 25//16
```

La trasformazione di una frazione nel corrispondente numero decimale si può fare con `Float64(<frazione>)`

Esempio:

```
Float64(25//16) dà 1.5625
```

Gli operatori `+` e `-` possono essere utilizzati per l'addizione o la sottrazione tra matrici numeriche della stessa dimensione.

Gli operatori `*` `/` e `^` possono essere utilizzati per il prodotto, la divisione e l'elevamento a potenza di matrici, rammentando che prodotto e divisione si possono fare se le colonne della prima matrice sono tante quante le righe della seconda e l'elevamento a potenza si può fare solo per le matrici quadrate.

Per ottenere prodotto, divisione ed elevamento a potenza membro a membro occorre far precedere all'operatore il simbolo punto (`.`).

Ovviamente `*` e `/` si utilizzano anche per moltiplicazioni e divisioni tra matrici e scalari, rammentando che, nelle divisioni, lo scalare può essere soltanto il secondo membro dell'operazione.

L'operatore `*` serve anche per concatenare stringhe.

2.4.2 Operatori di confronto

Servono per confrontare due valori e il risultato che restituiscono è un valore booleano. Sono i seguenti:

`==` uguale,
`!=` non uguale,
`<` minore,
`<=` minore o uguale,
`>` maggiore,
`>=` maggiore o uguale.

Gli operandi assoggettati al confronto devono avere lo stesso tipo.

2.4.3 Operatori logici

Forniscono come risultato un valore booleano e sono i seguenti:

`&&` per l'AND logico,
`||` per l'OR logico.

2.5 Interattività con l'utente

L'interfacciamento con l'utente avviene normalmente attraverso il terminale.

Esiste la possibilità di creare GUI utilizzando le librerie Gtk o il framework Tk ma non affronto queste cose in questo manualetto per dilettanti.

2.5.1 Output

Le funzioni per l'output sono `print` e `println` con la sintassi

```
print(<cosa_scrivere>)
```

```
println(<cosa_scrivere>)
```

dove `<cosa_scrivere>` può essere indicato con una stringa, una espressione matematica o il nome di una variabile che contiene il valore che vogliamo scrivere, anche combinati tra loro separati con una virgola (,).

La differenza tra le due è che la prima scrive senza andare a capo mentre la seconda scrive e va a capo.

Esempi:

```
println("ciao")
```

 scrive ciao e va a capo

```
println(5)
```

 scrive 5 e va a capo,

```
print(3*7.5)
```

 scrive 22.5 e non va a capo,

```
println("3 per 8 fa ", 3*8)
```

 scrive 3 per 8 fa 24 e va a capo,

data la variabile `x` contenente il valore 16,

```
print(x)
```

 scrive 16 e non va a capo,

```
print("la variabile x vale ", x)
```

 scrive la variabile x vale 16 e non va a capo.

Per formattare l'output dobbiamo ricorrere ad un modulo che vedremo.

2.5.2 Input

La funzione per l'input è `readline` con la seguente sintassi

```
readline()
```

L'esecuzione del programma resta sospesa fino a quando l'utente non ha inserito da tastiera il dato richiesto.

Eventuali istruzioni su che cosa l'utente debba inserire vanno impartite con un precedente messaggio scritto con `println()`.

Il dato inserito viene letto come stringa.

Se ciò che serve è un numero dobbiamo ricorrere alla funzione `parse()`, che abbiamo visto alla fine del Capitolo 4, per convertire immediatamente la stringa letta in numero.

Esempi:

```
x = readline() inserisce quanto digitato nella variabile x come stringa,  
y = parse(Float64, readline()) inserisce un numero, decimale a 64 bit, nella variabile y,  
println(2^(parse{Int, readline()})) se inseriamo 3 scrive 8.
```

2.6 Strutture di controllo

Come ogni altro linguaggio di programmazione, Julia ha dei comandi per condizionare l'esecuzione di certe istruzioni al verificarsi di determinate condizioni oppure per la ripetizione dell'esecuzione di una o più istruzioni.

2.6.1 Esecuzione condizionale

if

L'istruzione `if`, chiamata istruzione di esecuzione condizionale (in inglese `if` è il nostro `se`), ci dà modo di assoggettare l'esecuzione di un blocco di istruzioni al verificarsi di una determinata condizione: se la condizione è vera viene eseguito il blocco di istruzioni, altrimenti si prosegue l'esecuzione del programma saltando il blocco stesso.

La sintassi è la seguente

```
if <condizione>  
<istruzioni>  
end
```

dove `<condizione>` è una qualsiasi espressione che relaziona due valori attraverso operatori di confronto: se la condizione si verifica vengono eseguite la o le istruzioni indicate prima della parola chiave `end`, altrimenti si passa oltre.

L'istruzione `if` si presta anche all'esecuzione condizionale a due rami. Per ottenere questo dobbiamo abbinarla all'istruzione `else` con questa sintassi

```
if <condizione>  
<istruzioni>  
else  
<istruzioni>  
end
```

In questo caso se la condizione si verifica vengono eseguite le istruzioni contenute nel primo blocco, prima della parola chiave `else`, altrimenti vengono eseguite quelle contenute nel secondo blocco dopo `else` (altrimenti). In ogni caso proseguendo poi nell'esecuzione del resto del programma.

Possiamo infine gestire l'esecuzione condizionale a più rami abbinando a `if` l'istruzione `elseif` con questa sintassi

```
if <condizione>  
<istruzioni>  
elseif <condizione>  
<istruzioni>  
elseif <condizione>  
<istruzioni>  
else  
<istruzioni>  
end
```

2.6.2 Ripetizione

for

Si usa quando si vuole ripetere l'esecuzione di una istruzione o di un blocco di istruzioni per un numero definito di volte.

La sintassi per l'uso di questa istruzione in Julia è molto simile a quella che ritroviamo in Python:

```
for <range>
<istruzioni>
end
```

La situazione più semplice è quella che utilizza una sorta di contatore, una variabile creata al volo tradizionalmente indicata con `i`, che assuma un certo numero di valori:

```
for i = 1:5
println("Ciao")
end
```

scrive Ciao cinque volte.

Numero delle iterazioni ed elementi su cui eseguirle possono essere determinati utilizzando un qualsiasi oggetto che sia una sequenza.

Per esempio:

```
for x in "pippo"
println(x)
end
```

elenca su cinque righe le lettere che compongono la stringa "pippo"

```
for x in "pippo"
println("Ciao")
end
```

scrive Ciao cinque volte, quante sono le lettere che compongono la stringa "pippo"

while

Si usa per ripetere istruzioni fino a quando si verifica una certa condizione.

La sintassi è:

```
while <condizione>
<istruzioni>
end
```

Se la condizione è espressa attraverso l'uso di un contatore otteniamo gli stessi risultati che otteniamo con l'istruzione `for` vista prima

```
i = 1
while i <= 5
print "Ciao"
i = i + 1
end
```

scrive cinque volte la parola Ciao.

L'istruzione `while`, combinata con `break`, si presta ad interrompere un ciclo al verificarsi di un certo evento, come l'inserimento di una certa stringa da tastiera:

```
con
while true
x = readline()
if x == "fine"
break
end
end
```

viene richiesto un input fino a quando non si scrive la parola `fine`. Attenzione ai due `end`: il primo chiude il blocco di istruzioni `if` e il secondo chiude il blocco di istruzioni `while`.

2.7 Semplici programmi

Quanto visto finora ci consente di creare i nostri primi semplici script.

Cominciamo da questo, banalissimo, che chiede il nome all'utente e gli rivolge un saluto personalizzato:

```
println("Come ti chiami?")
nome = readline()
println("Ciao, ", nome, "!")
```

Quest'altro calcola la circonferenza e l'area di un cerchio chiedendone il raggio all'utente:

```
println("Digita il raggio di un cerchio")
r = parse(Float64, readline())
println("Per un cerchio di raggio ", r)
println("La circonferenza è ", r * 2 * pi)
println("e l'area è ", r^2 * pi)
```

L'esempio mi dà modo di ricordare che la costante matematica pi greco, richiamabile con pi, è l'unica costante matematica richiamabile senza aggiungere librerie a Julia di base.

Ora qualche cosa di più impegnativo, con uno script che chiede l'inserimento di una doppia serie di dati e calcola l'indice della correlazione esistente tra loro, determina l'equazione della retta di regressione lineare ed il relativo coefficiente di determinazione. Come dire che un po' di machine learning si può fare anche artigianalmente, senza ricorrere a chissà quali librerie. L'indice di correlazione di Pearson si calcola con la formula

$$r = \frac{\sum_i [(x_i - \bar{x})(y_i - \bar{y})]}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$$

Il coefficiente angolare della retta di regressione nella forma

$$y = a + bx$$

si calcola con la formula

$$b = \frac{\sum_i [(x_i - \bar{x})(y_i - \bar{y})]}{\sum_i (x_i - \bar{x})^2}$$

e l'intercetta si calcola con la formula

$$a = \bar{y} - b\bar{x}$$

Infine il coefficiente di determinazione è il quadrato dell'indice di correlazione.

Rammento che $\bar{x}$ e $\bar{y}$ sono rispettivamente la media delle x e la media delle y .

Lo script è riportato qui di seguito.

Ho utilizzato le indentazioni unicamente per rendere più visibili i blocchi di istruzioni e migliorare la lettura dello script. Le indentazioni non sono richieste da Julia.

```
sx = 0.0
sy = 0.0
n = 0
lx = Float64[]
ly = Float64[]
spscxy = 0.0
sqscx = 0.0
sqscy = 0.0
println("Inserisci i dati (f per finire)")
println("prima la x poi la y")
while true
    xx = readline()
    yy = readline()
    if xx == "f"
        break
    end
    push!(lx, parse(Float64, xx))
    push!(ly, parse(Float64, yy))
    spscxy += parse(Float64, xx) * parse(Float64, yy)
    sqscx += parse(Float64, xx)^2
    sqscy += parse(Float64, yy)^2
end
n = length(lx)
sx = sum(lx) / n
sy = sum(ly) / n
spscxy = spscxy / n
sqscx = sqscx / n
sqscy = sqscy / n
r = (spscxy - sx * sy) / sqrt((sqscx - sx^2) * (sqscy - sy^2))
println("Indice di correlazione di Pearson: ", r)
```

```

        end
        x = parse(Float64, xx)
        push!(lx, x)
        y = parse(Float64, yy)
        push!(ly, y)
        global sx = sx + x
        global sy = sy + y
        global n = n + 1
    end
mx = sx / n
my = sy / n
for i in 1:n
    global sqscx = sqscx + (lx[i] - mx)^2
end
for i in 1:n
    global sqscy = sqscy + (ly[i] - my)^2
end
for i in 1:n
    global spscxy = spscxy + (lx[i] - mx) * (ly[i] - my)
end
r = spscxy / (sqscx^(1/2) * sqscy^(1/2))
b = spscxy/sqscx
a = my - b * mx
println("dati:")
for i in 1:n
    println(lx[i], " ", ly[i])
end
println()
println("Coefficiente di correlazione:")
println(r)
println()
println("Retta di regressione:")
println("y = ", a , " + ", b, "x")
println()
println("Coefficiente di determinazione:")
println(r^2)

```

Innanzitutto abbiamo l'inizializzazione a valore zero delle variabili globali che dobbiamo avere a disposizione per far lavorare le formule (con 0.0 otteniamo una variabile destinata a contenere un tipo Float64 e con 0 otteniamo una variabile destinata a contenere un tipo Int64): `sx` e `sy` sono destinate a contenere la somma delle `x` e la somma delle `y`, `n` è destinata a contenere il numero delle coppie di dati `x` e `y`, `lx` e `ly` predispongono gli array destinati a contenere i valori `x` e `y`, `spscxy` conterrà la somma dei prodotti degli scarti delle `x` e delle `y` dalle loro medie (il numeratore delle formule per calcolare `r` e `b`), `sqscx` e `sqscy` conterranno le somme dei quadrati degli scarti delle `x` e delle `y` dalle loro medie, (per i denominatori delle formule per calcolare `r` e `b`).

Il successivo blocco `while` è destinato all'immissione dei dati, prevedendo il blocco all'inserimento della lettera `f` al posto dei due dati richiesti. I dati man mano inseriti vengono immessi nei rispettivi array, vengono sommati e contati.

Segue il calcolo delle medie.

Abbiamo poi tre cicli `for` in cui calcoliamo le sommatorie dei quadrati degli scarti e dei prodotti degli scarti.

Infine lo sviluppo delle formule e la presentazione dei risultati.

Nello sviluppo della formula per il calcolo della `r`, visto che ancora non conosciamo come si calcola la radice quadrata, ho utilizzato l'elevamento alla potenza $1/2$.

Chiudo la serie di questi primi esercizi con questo, nel quale costruiamo una matrice lavorando con la tastiera.

```

println("Creiamo una matrice.")
print("Quante colonne? ")
c = parse{Int, readline()}
print("Quante righe? ")
r = parse{Int, readline()}
M = []
colonna = []
println()
println("Colonna 1")
for ii = 1:r
    println("Elemento ", ii)
    push!(colonna, parse{Float64, readline()})
end
M = vcat(M, colonna)
for i = 2:c
    col = []
    println()
    println("Colonna ", i)
    for ii = 1:r
        println("Elemento ", ii)
        push!(col, parse{Float64, readline()})
    end
    global M = [M col]
end
println()
println(M)

```

Cominciamo chiedendo all'utente di quante colonne e di quante righe debba essere la matrice.

Inizializziamo poi una variabile globale, chiamata `M`, destinata a contenere la matrice.

Poi costruiamo la prima colonna: inizializzata la variabile `colonna` destinata a contenerla, avviamo un ciclo per chiedere all'utente gli elementi nel numero voluto.

Con la funzione `vcats` inseriamo questa prima colonna nella matrice, conferendo così alla matrice una struttura del numero di righe voluto.

Avviamo poi il ciclo per le successive colonne, con annidato il ciclo per inserirvi gli elementi. All'interno del ciclo le colonne che creiamo le chiamiamo con un nome diverso (`col`) per non confonderle con la variabile globale (`colonna`) che avevamo creato per la prima colonna.

Sempre all'interno del ciclo aggiungiamo ciascuna colonna alla matrice `M`.

Usciti dal ciclo stampiamo la matrice.

* * *

I programmi che abbiamo visto si salvano in file con estensione `.jl` e si lanciano nel terminale con il comando `julia` seguito dal nome del file.

Chi lavora in Linux può fare in modo che essi siano lanciabili semplicemente con il loro nome o cliccando sul loro nome, una volta resi eseguibili con `chmod` o agendo sulle `PROPRIETÀ/PERMESSI` nel gestore dei file. Per questo basta inserire nella prima riga, preceduto dai caratteri `#!`, il percorso e il nome dell'eseguibile `julia`.

Nel caso della mia installazione dovrei scrivere

```
#!/home/vittorio/.juliaup/bin/julia
```

Alla fine dello script, per fare in modo che il terminale non si chiuda senza darci modo di vedere il risultato delle elaborazioni, dovremo inserire l'istruzione `readline()`, magari preceduta dal messaggio «Premi Invio per uscire»

2.8 Funzioni

La funzione è una sezione del programma in cui è racchiusa una serie di istruzioni da eseguire e che vengono eseguite ogni volta che la funzione è chiamata.

Per semplificare la scrittura di un certo programma, potrebbe essere utile raggruppare alcune istruzioni all'interno del programma stesso, creando funzioni richiamabili, in modo da suddividere i compiti e da evitare di scrivere più volte le stesse cose.

La sintassi per definire una funzione è

```
function <nome_funzione> (<nome_parametro>, <nome_parametro>,...)
<istruzioni>
end
```

dove <nome_funzione> è il nome che intendiamo dare alla funzione, ed è il nome che useremo per richiamarla, <nome_parametro> è il nome del o dei parametri (dati) da inserire quando la richiamiamo.

Le <istruzioni> sono quelle relative alle elaborazioni da effettuare sui parametri per ottenere il risultato.

La funzione restituisce il valore dell'ultima espressione inserita nelle istruzioni. Con l'istruzione `return` prima di `end` possiamo stabilire noi che cosa debba restituire la funzione.

Possiamo stabilire il tipo dei parametri attraverso l'operatore `::` inserito tra il <nome_parametro> e il tipo voluto.

Possiamo stabilire il tipo del valore restituito facendo seguire all'elenco dei parametri l'operatore `::` a sua volta seguito dal tipo voluto.

La sintassi per chiamare una funzione ed eseguirla è

```
<nome_funzione> (<parametro>, <parametro>,...)
```

Esempi:

definita la seguente funzione

```
function saluta(nome)
print("Ciao ", nome)
end
```

con l'istruzione

```
saluta("Giuseppe") scriviamo Ciao Giuseppe.
```

definita la funzione

```
function area_triangolo(base::Float32, altezza::Float32)::Float64
base * altezza / 2
end
```

con l'istruzione

`area_triangolo(12, 6)` otteniamo il risultato `36.0` espresso come `Float64` e gli interi `12` e `6` che abbiamo inserito come parametri vengono letti come `Float32`.

Non sapendo ancora che esiste una funzione preconfezionata per calcolare il fattoriale di un numero scriviamola noi:

```
function fattoriale(n :: Int) :: Int128
if n == 0 return 1
else return n * fattoriale(n-1)
end
end
```

Si impone che il parametro da passare alla funzione sia un numero intero (se si passa un decimale il programma dà errore ed è giusto che sia così, in quanto il fattoriale esiste solo per i numeri interi) e che il risultato sia un intero a 128 bit, in modo da poter arrivare a calcolare il fattoriale di 33.

Dal momento che questa è una limitazione inaccettabile se vogliamo, per esempio, utilizzare la funzione per il calcolo combinatorio, conviene togliere le tipizzazioni e, quando si richiama la funzione, indicare il parametro `n` come `BigInt(n)` o semplicemente come `big(n)` in modo da portarci in precisione arbitraria e poter calcolare anche il fattoriale, per esempio, di 1345.

La funzione diventa

```
function fattoriale(n)
if n == 0 return 1
else return n * fattoriale(n-1)
end
end
e, richiamata in questo modo
fattoriale(BigInt(1345))
ci mostrerà un bel numero di 3626 cifre.
```

2.9 Funzioni preconfezionate di base

Julia ci offre tutta una serie di funzioni utili per tante cose senza che dobbiamo invocare alcuna libreria per poterle utilizzare.

Alcune le conosciamo già in quanto le ho utilizzate nei vari esempi.

Pensiamo alle funzioni `print()` e `println()` per l'output, alla funzione `readline()` per l'input e alla funzione `BigInt()` che abbiamo appena usato. Alle funzioni che abbiamo utilizzato per lavorare con array e matrici, come `push!()` e `vcut()`.

Moltissime altre ce ne sono e le troviamo descritte nella Parte II, Base, del manuale.

Qui elenco quelle di più ricorrente utilità in campo matematico. L'elenco completo si trova nel Paragrafo 45.2 del manuale.

2.9.1 Funzioni aritmetiche

Consentono la manipolazione di dati numerici.

`abs(n)` restituisce il valore assoluto del valore numerico n ,

`exp(n)` restituisce la potenza ennesima del numero e ,

`round(n)` restituisce, sotto forma di numero intero, la parte intera di n arrotondata,

`trunc(n)` restituisce, sotto forma di numero intero, la parte intera di n senza arrotondamenti,

`log(n)` restituisce il logaritmo naturale di n ,

`log10(n)` restituisce il logaritmo decimale di n ,

`log(b, n)` restituisce il logaritmo in base b di n ,

`pi` restituisce il valore della costante π ,

`sqrt(n)` restituisce la radice quadrata di n ,

`factorial(n)` restituisce il fattoriale di n ,

`hypot(x, y)` restituisce l'ipotenusa con cateti x e y ,

`lcm(n, n, n, ...)` restituisce il minimo comune multiplo dei numeri indicati.

Per la potenza ennesima di x possiamo usare la combinata `exp(log(x)*n)`.

Per la radice ennesima di x possiamo usare la combinata `exp(log(x)/n)`.

2.9.2 Funzioni trigonometriche

Esprimendo n in radianti abbiamo le funzioni dirette `sin(n)`, `cos(n)` e `tan(n)`.

Esprimendo n in gradi abbiamo le funzioni dirette `sind(n)`, `cosd(n)` e `tand(n)`.

Le funzioni inverse `asin(n)`, `acos(n)` e `atan(n)` restituiscono l'angolo in radianti.

Le funzioni inverse `asind(n)`, `acosd(n)` e `atand(n)` restituiscono l'angolo in gradi.

`deg2rad(n)` converte gradi in radianti.

`rad2deg(n)` converte radianti in gradi.

2.9.3 Funzioni relative ad array numerici

`length(<array>)` restituisce il numero degli elementi,

`minimum(<array>)` restituisce il valore minimo contenuto,

`maximum(<array>)` restituisce il valore massimo contenuto.

2.10 Dot syntax e Funzioni anonime

Il linguaggio Julia possiede una particolarità, che non si trova in altri linguaggi di programmazione, riguardante i calcoli che coinvolgono array: attraverso l'uso di un punto (.) ben collocato insieme a operatori o richiami di funzione si ottiene quella che possiamo chiamare vettorizzazione di un'operazione matematica.

Nel Capitolo 5 abbiamo visto come si possano utilizzare normalmente, con quali effetti e con quali limiti, i normali operatori aritmetici tra array o tra array e scalari.

Ci rimane qualche problema con l'elevamento a potenza di un array, possibile con il semplice operatore ^, a patto che la base sia una matrice quadrata ed ottenendo come risultato l'elevamento a potenza matriciale.

Già allora abbiamo visto che per ottenere l'elevamento a potenza membro a membro occorre far precedere all'operatore ^ un punto (.) ed abbiamo così anticipato la tecnica della dot syntax.

In questo modo non esiste più l'esigenza che la base della potenza sia una matrice quadrata, ma essa può essere una matrice qualsiasi o un array monodimensionale.

Così, in presenza dell'array A, con

```
A .^ 3
```

otteniamo un nuovo array i cui elementi sono il cubo degli elementi dell'array A.

Allo stesso modo possiamo sommare uno scalare a un array o sottrarre uno scalare da un array:

```
A .+ <scalare> oppure A .- <scalare>
```

La cosa si fa ancor più interessante se estendiamo il discorso alle funzioni.

Tutte le funzioni che abbiamo visto nel precedente Capitolo 9, e quante ne possiamo costruire noi, se prevedono come argomento un numero, possiamo utilizzarle passando come argomento un array, di qualsiasi tipo e dimensione, ottenendo un nuovo array i cui elementi sono il risultato dell'applicazione della funzione a ciascun elemento dell'array di partenza.

Così, in presenza dell'array A, con

```
<nome_funzione>.(A)
```

otteniamo un nuovo array i cui elementi sono il risultato dell'applicazione della funzione a ciascun elemento dell'array di partenza.

Altra particolarità di Julia è quella delle funzioni anonime, utili per semplificare la generazione di un array attraverso calcoli applicati agli elementi di un altro array.

La sintassi per generare una funzione anonima con argomento x è la seguente

```
x -> <espressione>
```

Così con

```
x -> x^(1/2)
```

generiamo una funzione anonima che calcola la radice quadrata di x.

L'uso della funzione anonima torna utile ogniqualvolta dobbiamo passarla come argomento ad un'altra funzione.

Esiste, per esempio, la funzione map, destinata a generare un insieme di dati applicando una certa funzione ad un altro insieme di dati. La sua sintassi è

```
map(<funzione>, <dati>)
```

Per generare un array formato dalle radici quadrate dell'insieme di dati contenuto nell'array A basta fare

```
map(x -> x^(1/2), A)
```

2.11 Librerie standard

Una libreria è una raccolta di funzioni.

Le funzioni che abbiamo visto finora fanno parte dalla grande libreria di base e possiamo richiamarle direttamente.

Una volta installato Julia abbiamo però a disposizione tantissime altre funzioni che sono raccolte in altre librerie.

Alcune di queste librerie sono disponibili senza installare alcunché di aggiuntivo a Julia. Esse sono già installate con Julia e, per servircene, dobbiamo semplicemente richiamarle con la sintassi

```
using <nome_libreria>.
```

Queste sono le librerie raggruppate nella Standard Library e le troviamo descritte nella Parte III del Manuale di Julia.

Vi sono poi moltissime altre funzioni che fanno parte di librerie che non vengono installate con Julia (salvo ricorrere a Julia-Pro): sono i così detti Packages.

Per utilizzare queste funzioni dobbiamo preventivamente installare il package di cui fanno parte.

In questo capitolo presento sommariamente alcune librerie che fanno parte della Standard Library.

2.11.1 Printf

Contiene la funzione utile per formattare l'output.

Per utilizzarla occorre l'istruzione

```
using Printf
```

Abbiamo così a disposizione la direttiva di formattazione

```
"%.<cifre_decimali>f"
```

utilizzabile con la funzione `@printf` per stabilire le cifre decimali da scrivere arrotondando l'ultima.

Per esempio:

```
print(7.168 * 4.68) scrive 33.54624
```

```
@printf "%.2f" 7.168*4.68 scrive 33.55
```

```
@printf "%.5f" pi scrive 3.14159
```

2.11.2 LinearAlgebra

Contiene funzioni per l'algebra lineare.

Abbiamo visto che già con la dotazione base di Julia abbiamo a disposizione funzioni per l'aritmetica delle matrici (addizione, sottrazione, prodotto, divisione).

In questa libreria troviamo funzioni per l'algebra lineare e, per utilizzarla, occorre l'istruzione

```
using LinearAlgebra
```

Abbiamo così a disposizione, tra le altre, le funzioni

`det(M)` per calcolare il determinante della matrice `M`,

`inv(M)` per calcolare la matrice inversa di `M`.

2.11.3 Statistics

Contiene funzioni statistiche.

Per utilizzarla occorre l'istruzione

```
using Statistics
```

Tra le altre, abbiamo a disposizione le funzioni

`mean(<array>)` per calcolare la media della serie contenuta in un array,

`std(<array>)` per calcolare la deviazione standard campionaria della serie contenuta in un array,

`var(<array>)` per calcolare la varianza campionaria della serie contenuta in un array,

`cor(<vettore_x>, <vettore_y>)` per calcolare l'indice di correlazione di Pearson tra una serie di `x` e una serie di corrispondenti `y` contenute in due vettori (array verticali).

2.12 Orientamento agli oggetti

Non si può dire che Julia sia un linguaggio che agevola la programmazione per oggetti e non consiglio a un dilettante di provarci.

Tuttavia in Julia è tutto un oggetto, anche se, abituati ad altri linguaggi, non ce ne accorgiamo, visto che Julia utilizza una sintassi tutta sua per lavorare con gli oggetti.

Generalmente, quando si richiama la funzione membro di un oggetto, si usa la sintassi `<nome_oggetto>.<funzione_con_relativi_parametri>`.

In Julia la sintassi è `<nome_funzione>(<nome_oggetto>, <parametri_della_funzione>)`.

Prendiamo, per esempio, la funzione con cui si aggiunge un elemento a un vettore.

In Julia la utilizziamo così

```
push!(<vettore>, <elemento>)
```

e in Python, dove avremmo una lista al posto del vettore, la utilizziamo così, come succederebbe in Java e C++

```
<lista>.append(<elemento>)
```

2.13 Lavorare con file

Julia ci offre tante modalità per lavorare con i file, alcune semplificanti ma pericolosissime per la salute dei file stessi se non usate con la dovuta attenzione.

Qui rammento la più sicura, molto simile a quella che ritroviamo in altri linguaggi, basata sulla funzione `open()` con la quale creiamo un oggetto file dotato di funzioni per scriverne, aggiornarne o leggerne il contenuto.

La sintassi è la seguente

```
f = open(<nome_file>, <modalità>)
```

dove `f` è un qualsiasi nome che diamo all'oggetto file,

`<nome_file>` è una stringa che indica il percorso al file,

`<modalità>` è una stringa che può assumere i seguenti valori:

"w" se vogliamo scrivere sul file, creandolo se non c'è o sostituendolo,

"a" se vogliamo aggiungere elementi ad un file esistente senza sostituirlo,

"r" se vogliamo leggere il file.

Le più importanti funzioni membro del nostro oggetto file sono

`write(<oggetto_file>, <stringa>)` per scrivere una stringa nel file (per andare a capo occorre terminare la stringa con `\n`),

`read(<oggetto_file>, String)` per leggere tutto il file come unica stringa,

`readline(<oggetto_file>)` per leggere una riga del file (una volta letta una riga ci si posiziona sulla riga successiva e occorre ripetere il comando per leggerla),

`readlines(<oggetto_file>)` per leggere tutto il file e inserirne il contenuto in un array i cui elementi sono costituiti da una riga del file.

Per rendere operativo ciò che abbiamo fatto sul file e chiuderlo posizionandosi sul primo elemento occorre utilizzare il metodo

```
close(<oggetto_file>)
```

Esempi:

```
con f = open("/home/vittorio/Documenti/prova", "w")
```

creo il file prova nella posizione indicata dal percorso,

```
con write(f, "Pippo\n")
```

 scrivo la sua prima riga con il nome Pippo,

```
con close(f)
```

 chiudo il file, confermando quanto scritto in esso,

```
con f = open("/home/vittorio/Documenti/prova", "a")
```

apro il file prova per aggiungere altri elementi

```
con write(f, "Pluto\n", "Paperino\n")
```

aggiungo altre due righe,

```
con close(f)
```

 chiudo il file, confermando quanto aggiunto,

```
con f = open("/home/vittorio/Documenti/prova", "r")
```

apro il file prova per leggerne il contenuto
con `readline(f)` leggo la prima riga e, se ripeto il comando subito dopo, leggo la seconda e così via

con `close(f)` chiudo il file e riposiziono tutto.

Se riapro il file in lettura,

con `readlines(f)` ottengo l'array

```
"Pippo"
```

```
"Pluto"
```

```
"Paperino"
```

con `read(f, String)` otterrei la stringa

```
"Pippo\nPluto\nPaperino\n"
```

Il metodo `write` può scrivere una sola stringa per volta, per cui se vogliamo scrivere più stringhe dobbiamo separarle con una virgola e se vogliamo scrivere numeri dobbiamo lavorare di conversione di tipo.

Per esempio, con il comando

```
write(f, "3 moltiplicato 2 fa ", string(3 * 2), "\n")
```

scriveremmo nel nostro file la riga

```
3 moltiplicato 2 fa 6
```

Per elaborare file di dati su più righe e più colonne, usualmente in formato `.csv`, dobbiamo ricorrere al package `CSV`.

2.14 Packages

Nel paragrafo 1.2 abbiamo parlato del repository di Julia, che contiene moltissimi pacchetti aggiuntivi per arricchire le potenzialità del linguaggio base.

Per utilizzare un package dobbiamo innanzi tutto installarlo

L'installazione avviene utilizzando la shell di Julia in modalità `Pkg`.

Per entrare in questa modalità inseriamo al prompt di Julia una parentesi quadra chiusa `]`.

Il prompt si trasformerà così da `julia>` a `pkg>`.

Da qui, con il comando `?` otteniamo l'elenco di tutti i comandi previsti per la gestione dei pacchetti con una sommaria descrizione del loro scopo e con il comando `?` seguito dal nome di un comando otteniamo la descrizione completa della sintassi per utilizzare il comando stesso.

I comandi di più ricorrente utilizzo sono:

`add <nome_pacchetto>` per installare un pacchetto,

`rm <nome_pacchetto>` per disinstallare un pacchetto,

`st` per vedere l'elenco dei pacchetti installati.

La disinstallazione con `rm` disattiva il pacchetto ma non rimuove i file che costituiscono il pacchetto dal disco e li tiene in cache al fine di velocizzare una eventuale reinstallazione.

Esiste il comando `gc --all` (che sta per garbage collector) che ci consente di liberare spazio di memoria occupato da rimasugli di pacchetti disinstallati e non più usati per un certo periodo (mi pare di aver capito attorno ai 30 giorni).

E' importante tener presente queste cose in quanto i pacchetti di Julia tendono ed essere alquanto pesanti in termini di occupazione di spazio su disco.

Per uscire dalla modalità `Pkg` usiamo il tasto `BACKSPACE` o la combinazione `CTRL - C`.

Per poter usare un pacchetto dobbiamo inserire nello script, prima di usarlo, il comando `using <nome_pacchetto>`

La prima volta che si usa il pacchetto, e solo in questa occasione, occorre attendere qualche tempo affinché si compia la precompilazione.

Nei prossimi capitoli parleremo di alcuni di questi pacchetti.

Qui vorrei soffermarmi sulla possibilità di utilizzare in Julia funzioni del linguaggio Python attraverso il package **PyCall**, importando il modulo Python che le contiene.

Per default questo pacchetto, alla sua installazione, si collega all'ecosistema Python3 presente sul computer e, ovviamente, i moduli richiamati devono essere installati.

Dopo la dichiarazione `using PyCall`, l'importazione del modulo che interessa avviene creando un oggetto che contiene le funzioni del modulo stesso con
`<nome_oggetto> = pyimport(<stringa_nome_modulo>)`
dopo di che ogni funzione è richiamabile con
`<nome_oggetto>.<nome_funzione>`
come se lavorassimo in Python.

Se, per esempio, vogliamo calcolare l'integrale definito tra 0 e 2 della funzione x^2 utilizzando il sotto-modulo `integrate` di `scipy`, basta scrivere in uno script da memorizzare quanto segue

```
using PyCall
i = pyimport("scipy.integrate")
println(i.quad(x->x^2, 0, 2))
```

e con questo misto di linguaggio Julia e Python otteniamo il risultato in questa tupla
(2.666666666666667, 2.960594732333751e-14)

il cui primo valore è quello dell'integrale e il secondo è l'errore di approssimazione del calcolo.

Il package `PyCall` dà accesso a tutto l'ecosistema Python che abbiamo installato sul computer.

Per certi moduli Python esistono specifici packages Julia di interfacciamento che rendono ancor più agevole l'integrazione tra i linguaggi. Penso, per esempio ai packages **PyPlot** e **SymPy**.

Di notevole importanza quest'ultimo, grazie al quale possiamo arricchire Julia della potenzialità del calcolo simbolico di Python, oggi come oggi più ricca di quella di Julia.

Teniamo comunque presente che quanto abbiamo descritto in questo manuale può tranquillamente essere praticato su un computer non particolarmente dotato di risorse in termini di velocità del processore e di RAM.

Se utilizziamo pacchetti aggiuntivi al sistema di base qui trattato teniamo presente che, specialmente i pacchetti di grafica dell'ecosistema Julia, sono particolarmente esosi di risorse e, con computer datati o dotati di risorse limitate, potremmo avere difficoltà, soprattutto in termini di tempo di esecuzione dei nostri script.

2.15 Esercizi conclusivi

Per esemplificare un po' tutto ciò che abbiamo visto anche negli ultimi Paragrafi propongo un paio di script.

Il primo serve per risolvere un sistema di equazioni lineari di qualsiasi dimensione.

```
using LinearAlgebra
println("Risoluzione di un sistema di equazioni lineari.")
print("Quante equazioni? ")
n = parse{Int, readline()}
M = Float64[]
colonna = Float64[]
println()
println("Coefficienti delle X1")
for ii = 1:n
    println("equazione ", ii)
    push!(colonna, parse{Float64, readline()})
end
M = vcat(M, colonna)
for i = 2:n
    col = Float64[]
    println()
    println("Coefficienti delle X", i)
    for ii = 1:n
        println("equazione ", ii)
        push!(col, parse{Float64, readline()})
    end
end
```

```

    global M = [M col]
end
println()
println("Termini noti:")
N = []
for i = 1:n
    push!(N, parse(Float64, readline()))
end
x = inv(M) * N
println()
println("Soluzioni:")
for i = 1:n
    print("X", i, " = ")
    println(x[i])
end

```

La risoluzione del sistema è ottenuta attraverso la nota formula

$$X = M^{-1}N$$

dove X è il vettore delle soluzioni, M è la matrice dei coefficienti delle incognite (M^{-1} ne è l'inversa) e N è il vettore dei termini noti.

La costruzione della matrice, semplificata dal fatto che ora dobbiamo costruire una matrice quadrata, avviene come abbiamo visto in un esercizio nel Capitolo 8.

La novità è rappresentata dall'applicazione della formula per ottenere la soluzione e dalle cinque righe finali per scriverle.

Per disporre della funzione di inversione della matrice abbiamo importato la libreria standard LinearAlgebra (nelle ultime versioni di Julia sarebbe comunque disponibile).

Con quest'altro script possiamo risolvere un'equazione, calcolando le radici di equazioni anche di grado elevato.

Se abbiamo installato il package PyCall lo script potrebbe essere il seguente

```

using PyCall
s = pyimport("sympy")
println("Ricerca delle radici di un'equazione quasiasi")
println("Inserisci la parte sinistra di una equazione uguagliata a zero")
println("(esprimere utilizzando gli operatori ^ * / + -)")
espressione = readline()
x = s.symbols("x")
e = s.sympify(espressione)
soluzioni = s.solve(e, x)
for i = 1:length(soluzioni)
    println("soluzione ", i, ": ", s.N(soluzioni[i]))
end

```

Se abbiamo installato solo SymPy o se vogliamo comunque utilizzare quello, lo script potrebbe essere il seguente

```

using SymPy
println("Ricerca delle radici di un'equazione quasiasi")
println("Inserisci la parte sinistra di una equazione uguagliata a zero")
println("(esprimere utilizzando gli operatori ^ * / + -)")
espressione = readline()
x = symbols("x")
e = sympify(espressione)
soluzioni = solve(e, x)
for i = 1:length(soluzioni)
    println("soluzione ", i, ": ", N(soluzioni[i]))
end

```

In questo caso siamo in pieno calcolo simbolico e ricorriamo, in un modo o in un altro, alla potenza di Python.

Le istruzioni evidenziate in rosso sono nel linguaggio di SymPy e, come si vede, convivono con istruzioni in linguaggio Julia per arrivare a risultati strabilianti con uno script così piccolo.

Lavorando in questo modo, tuttavia, si nota un po' meno la decantata velocità di Julia.

3 Grafica per GUI

Sono convinto che un linguaggio che ha le finalità di Julia può benissimo dare il meglio di sé senza interfacciarsi graficamente con l'utente: più che una semplificazione, per fare certe cose, l'interfaccia grafica può essere un impiccio.

Salvo casi in cui l'interfaccia grafica sia unificata in un'applicazione in cui si accolgano dati dall'utente e si rappresentino in tempo reale gli effetti grafici del cambiamento dei dati. Per cose di questo tipo esiste un pacchetto, che si chiama Makie, molto impegnativo sia per l'hardware sia per chi lo usa, che lasciamo ad esperti professionisti.

Per dilettanti come noi, fino a qualche tempo fa, Julia prevedeva la possibilità di creare interfacce grafiche utilizzando il pacchetto Tk.jl, che metteva a nostra disposizione tutto l'armamentario del software Tk, come avviene in Python con il modulo tkinter.

Visto che ormai tutti sono convinti che le GUI fatte con Tk siano vecchie e brutte (mentre fanno benissimo il loro mestiere con poca spesa), nel mondo Python un volenteroso ha creato customtkinter per far sì che le GUI tkinter siano più moderne mentre, nel mondo Julia, è stato deprecato l'uso del pacchetto Tk.

Il fatto è che le alternative, a parte il citato pacchetto Makie, con cui si possono anche realizzare GUI semplici, utilizzando un carro armato per schiacciare una mosca, sono tutte abbastanza complicate e utilizzano le librerie Qt, GTK con pacchetti che troviamo nel repository di Julia (QML.jl, GTK.jl).

A questo punto, per noi dilettanti, la via migliore è quella di ricordarci che da Julia possiamo avere accesso ai moduli Python attraverso il pacchetto PyCall e, per questa via, costruire GUI utilizzando i moduli tkinter o customtkinter di Python.

Un piccolo esempio.

Con questo script creiamo una finestra utilizzando il tool customtkinter di Python.

```
using PyCall
tk = pyimport("customtkinter")
r = tk.CTk()
r.title("PROVA")
r.config(width = 300, height = 200, bg = "yellow")
r.mainloop()
```

con questo risultato



Attenzione al fatto che lo script deve girare su un interprete/compiler Julia, pertanto i doppi apici, che se fossimo in Python potrebbero anche essere semplici apici, qui devono essere proprio doppi (in Julia, infatti, tra semplici apici ci può stare un solo carattere, non una stringa).

Con quest'altro script creiamo una piccola applicazione che chiede all'utente il nome per salutarlo

```

using PyCall
function saluta()
nome = e.get()
l2.configure(text="Ciao, " * nome * "!")
end
tk = pyimport("customtkinter")
r = tk.CTk()
r.title("SALUTO")
r.config(bg = "PaleGreen")
l1 = tk.CTkLabel(r, text = "Come ti chiami?", fg_color = "PaleGreen")
l1.grid(column = 0, row = 0, sticky = "w", padx = 4)
e = tk.CTkEntry(r)
e.grid(column = 1, row = 0, padx = 4, pady = 4)
b = tk.CTkButton(r, text = "SALUTA", width = 80, command = saluta)
b.grid(column = 0, row = 1, columnspan = 2, pady = 4)
l2 = tk.CTkLabel(r, height = 30, text = "", fg_color = "PaleGreen", text_color = "red", font = ("Amigos",20))
l2.grid(column = 0, row = 2, columnspan = 2, pady = 4)
r.mainloop()

```

Come si vede, anche scrivendo istruzioni tipiche del modulo customtkinter si deve rispettare la sintassi di Julia (oltre all'attento uso del doppio apice per le stringhe abbiamo l'operatore `*` per concatenare stringhe).

Il risultato è il seguente



A sinistra l'applicazione all'apertura e a destra l'applicazione dopo aver inserito il nome della persona da salutare e cliccato sul pulsante SALUTA.

4 Grafica per la rappresentazione di dati

Tutti sappiamo quanto aiuti la grafica a rendere meglio leggibili e interpretabili i dati.

In Statistica linee, torte e canne d'organo rendono immediata la comprensione di trend, di ripartizioni e di raffronti come nessuna tabella può fare.

In Matematica i grafici di funzione rendono l'idea di sintesi del comportamento di una funzione matematica come nessun indicatore analitico può fare.

Julia ci mette a disposizione moltissimi pacchetti originali per fare queste cose e ci consente di utilizzare con estrema facilità gli strumenti aventi le stesse finalità che troviamo nell'ecosistema Python.

Se siamo votati a Julia in esclusiva e non abbiamo Python sul nostro sistema possiamo scegliere tra i moltissimi strumenti originali.

Se abbiamo sul nostro sistema anche Python arricchito degli strumenti per le ricerche scientifiche e la scienza dei dati possiamo creare dei mix molto efficienti tra i due linguaggi con il vantaggio di poter scegliere il meglio dell'uno e dell'altro.

4.1 Plots

Plots è il package originale dell'ecosistema Julia che ha sostituito il package Winston, rispetto al quale è molto più ricco.

E' diventata la libreria di visualizzazione più diffusa e versatile dell'ecosistema Julia.

Ha il pregio di creare grafici potendo utilizzare diversi motori grafici, senza dover modificare il codice.

I motori grafici che può utilizzare sono:

- . GR quello di default, molto veloce, ideale per uso normale,
- . PlotlyJS per grafici interattivi,
- . PythonPlot basato su Matplotlib di Python,
- . PGFPlotsX con qualità tipografica per LaTeX,
- . UnicodePlots per grafici direttamente nel terminale.

Installando Plots si installa anche il motore GR.

Per utilizzare altri motori grafici occorre installare, oltre a Plots, i relativi package, che hanno il nome indicato nell'elenco appena visto.

Direi che per principianti basta e avanza che ci si accontenti del motore di default GR.

Anche perché per grafici interattivi è molto meglio il package di cui parlo nel prossimo Capitolo e per grafici basati su Matplotlib di Python ci dà meno problemi agire come vedremo nel prossimo Paragrafo 4.3.

Per il resto si tratta di perfezionamenti da professionisti.

Per usare Plots occorre dichiararne l'intenzione all'inizio dello script e subito dopo dichiarare il motore grafico preferito, con questa sintassi:

```
using Plots
gr()
```

La seconda indicazione non è necessaria se vogliamo usare il motore GR ma è obbligatoria se vogliamo usare un altro motore.

All'indirizzo <https://docs.juliaplots.org/stable/> troviamo un ottimo manuale di Plots in lingua inglese.

Qui presento una piccola rassegna delle funzioni principali di questo potente strumento grafico.

4.1.1 plot()

La funzione `plot()`, per default, traccia un grafico a linea.

Gli argomenti necessari sono gli array contenenti i dati di base per il tracciamento.

In presenza di un solo array `y` dei dati per le ordinate, con `plot(y)` tracciamo una linea che congiunge i dati dell'array in corrispondenza delle ascisse 1, 2, 3, 4,

In presenza di due array `x` per le ascisse e `y` per le ordinate, con `plot(x,y)` tracciamo una linea che congiunge le corrispondenze tra le `x` e le `y`.

Tra le parentesi tonde, oltre agli argomenti, possiamo inserire attributi per il grafico. Ricordo qui solo gli attributi principali:

- . attributi per titoli e etichette:

- `title` per esprimere con una stringa il titolo del grafico,
- `label` per etichettare il grafico con una stringa (con valore `false` l'etichetta scompare),
- `xl` per esprimere con una stringa l'etichetta dell'asse delle ascisse,
- `yl` per esprimere con una stringa l'etichetta dell'asse delle ordinate,

- . attributi per modificare l'aspetto visivo della linea o del tipo di grafico:

- `color` per esprimere il colore della linea con sintassi `:<nome_colore>`,
- `lw` per esprimere lo spessore della linea con un numero,
- `ls` per esprimere lo stile della linea con sintassi `:solid`, `:dash`, `:dot`, `:dashdot`,
- `st` per definire il tipo di grafico con possibile scelta tra i principali
 - `:path` per la linea, che è il default,
 - `:scatter` per i punti,
 - `:bar` per un istogramma a barre.

- . altri attributi:

- `grid` per mostrare o nascondere la griglia di sfondo (`true` o `false`),
- `size` per dimensionare il grafico in pixel come tupla (es. `size=(800, 600)`),
- `bg` per esprimere un colore di sfondo del grafico con sintassi `:<nome_colore>`.

* * *

Con la funzione `plot!` possiamo realizzare la modifica incrementale di un grafico, cioè aggiungere un grafico ad uno già esistente.

* * *

Le funzioni `plot` e `plot!` si prestano egregiamente per rappresentare grafici di funzioni matematiche, accettando come argomenti la funzione da rappresentare e il campo di variazione.

Se la funzione è riconosciuta da Julia, basta scriverne il nome:

`plot(sin, -pi, pi)` traccia il grafico della funzione $y = \sin(x)$ nell'intervallo di ascisse $-\pi: \pi$

Se la funzione non è riconosciuta da Julia possiamo crearla noi come funzione anonima:

`plot(x -> x^2, -5, 5)` traccia il grafico della funzione $y = x^2$ nell'intervallo di ascisse $-5:5$.

Per definire sia l'intervallo delle ascisse sia quello delle ordinate da visualizzare occorre usare gli attributi `xlims` e `ylims`, uguagliandoli alla tupla contenente gli estremi degli intervalli.

Con qualche complicazione in più possiamo creare grafici tridimensionali per funzioni di due variabili, utilizzando il comando `surface` applicato agli array relativi ai valori delle x e delle y e alla funzione $f(x, y)$.

* * *

Per salvare un grafico prodotto con `plot()` possiamo assegnarlo a una variabile e poi salvarlo con la funzione

`savefig(<nome_variabile>, <nome_grafico>)`

dove `<nome_grafico>` è il nome da dare al grafico, completo di percorso, con estensione `.png`, `.pdf` o `.svg`, a seconda del formato voluto.

* * *

Possiamo produrre il grafico dalla REPL di Julia inserendo via via le istruzioni.

Se vogliamo salvare lo script in un file da lanciare quando serve con l'interprete Julia, dobbiamo avere l'avvertenza di inserire, alla fine dello script, le istruzioni

`gui()`

`readline()`

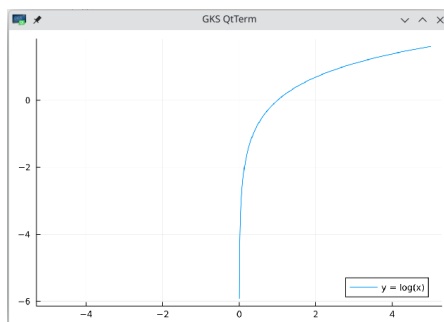
La prima per forzare la produzione del grafico, la seconda per farlo rimanere visibile fino a quando non si preme un tasto.

* * *

Con questo comando nella REPL

`plot(log, -5, 5, label = "y = log(x)")`

otteniamo



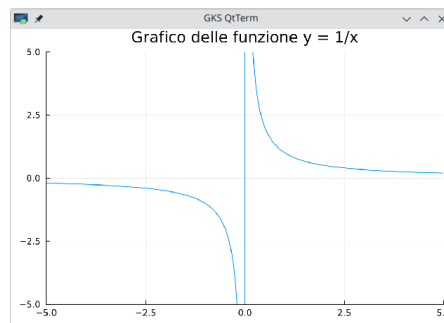
dove abbiamo l'asse delle ascisse visualizzato tra -5 e 5 e quello delle ordinate visualizzato a scelta del programma alla dimensione utile per la lettura del grafico, scartando automaticamente le zone prive di dati (zona superiore) o prive di dati significativi (zona inferiore con l'asintoto).

Se passiamo il comando

`plot(x->1/x, -5, 5, title = "Grafico delle funzione y = 1/x")`

otteniamo un grafico illeggibile e dobbiamo governare noi anche l'asse delle ordinate in questo modo

```
plot(x->1/x, xlims=(-5, 5),ylims=(-5, 5), title = "Grafico delle funzione y = 1/x", legend=false)
ottenendo
```



Con questo script tracciamo il grafico tridimensionale della funzione

$$z = x^2 - y^2$$

detta Funzione a sella (paraboloide iperbolico)

```
using Plots
```

```
f(x, y) = x^2 - y^2
```

```
x = range(-2, 2, length=30)
```

```
y = range(-2, 2, length=30)
```

```
surface(x, y, f, title="Grafico della funzione Sella",xl="X", yl="Y", zl="Z")
```

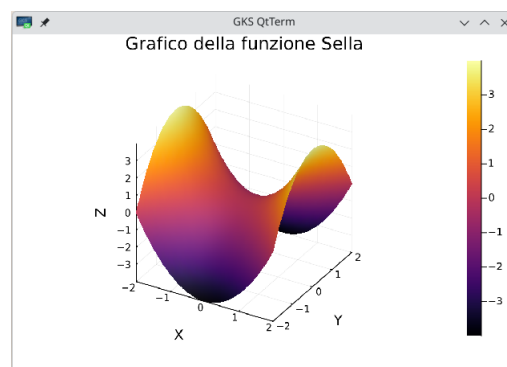
```
gui()
```

```
readline()
```

Il piccolo trucco sta nel ridefinire la funzione z come funzione di due variabili $f(x, y)$ e definire gli intervalli per gli assi x e y .

Passando poi, insieme agli array x e y la funzione f a `surface`, Julia valuta la funzione su ogni combinazione di x e y .

Lanciando lo script con Julia otteniamo



4.1.2 scatter()

La funzione `scatter()` disegna sul piano i punti che rappresentano i dati, senza unirli con una linea.

Possiamo ottenere la stessa cosa con la funzione `plot()`, definendo opportunamente l'attributo `st`.

Gli attributi di abbellimento del grafico sono per lo più gli stessi della funzione `plot()`.

Notevole l'attributo `smooth`, dando al quale il valore `true` insieme al grafico dei punti viene tracciata la retta di regressione.

* * *

Dati gli array

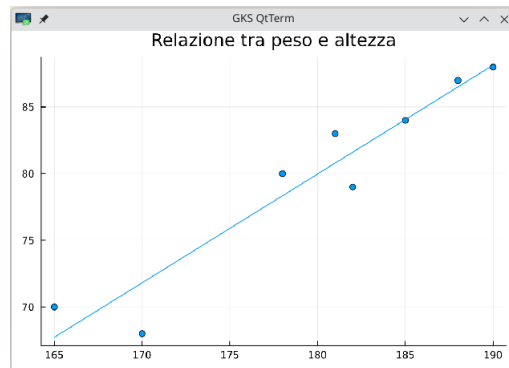
```
x = [165,170,178,181,182,185,188,190]
```

```
y = [70,68,80,83,79,84,87,88]
```

che relazionano l'altezza (x) e il peso (y) di un piccolo campione di persone, con questo comando nella REPL

```
scatter(x, y, title = "Relazione tra peso e altezza", smooth=true, legend=false)
```

otteniamo



4.1.3 bar()

La funzione `bar()` fa apparire il valore dell'ordinata corrispondente ad una ascissa con una barra verticale.

Possiamo ottenere la stessa cosa con la funzione `plot()`, definendo opportunamente l'attributo `st`.

Gli attributi di abbellimento del grafico sono per lo più gli stessi della funzione `plot()`.

Il grafico a barre è utile per far corrispondere una serie di dati rappresentati in un array `y` a delle categorie classificatorie rappresentate in un array di stringhe `x`.

L'importo delle vendite delle nostre filiali di Milano, Roma, Torino e Genova è

Milano	678
Roma	456
Torino	525
Genova	367

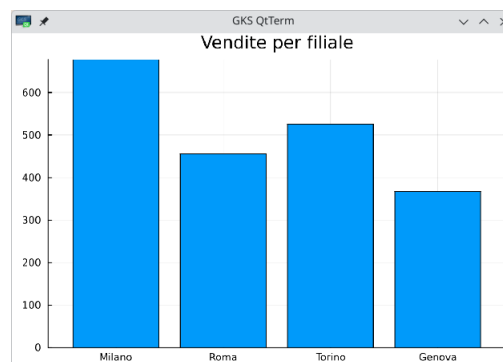
Scrivendo nella REPL

```
x = ["Milano", "Roma", "Torino", "Genova"]
```

```
y = [678, 456, 525, 367]
```

```
bar(x, y, title = "Vendite per filiale", legend = false)
```

otteniamo



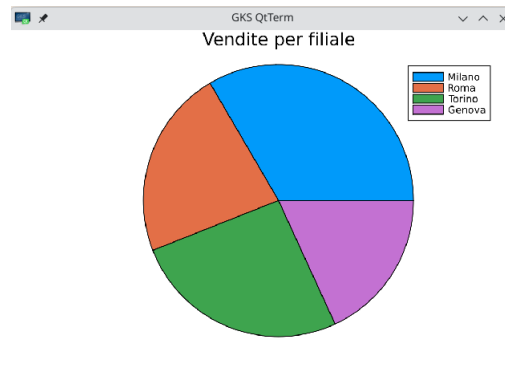
4.1.4 pie()

La funzione `pie()` produce una torta le cui fette, differenziate per colore, sono proporzionali all'entità dei valori delle ordinate.

E' utile, come il grafico a barre, per far corrispondere una serie di dati rappresentati in un array `y` a delle categorie classificadorie rappresentate in un array di stringhe `x`.

* * *

Con le stesse array sulle vendite per filiale dell'esempio precedente, scrivendo nella REPL `pie(x, y, title = "Vendite per filiale")` otteniamo



4.2 Makie

Anche Makie è un package originale dell'ecosistema Julia, probabilmente il migliore e il più completo.

L'utilizzo del pacchetto è alquanto impegnativo, sia sul piano della difficoltà di apprendimento sia sul piano delle risorse: se non abbiamo particolari inclinazioni informatiche è meglio che non lo utilizziamo e se lo utilizziamo su computer non particolarmente dotati dobbiamo pazientare sui tempi di risposta.

All'indirizzo <https://makie.juliaplots.org/stable/> è comunque discretamente documentato in una facile lingua inglese ed ho ritenuto utile citarlo per chi voglia provare a cimentarsi con esso utilizzando questa documentazione, di una completezza difficilmente riscontrabile per altri pacchetti dell'ecosistema Julia.

Lo possiamo installare attraverso uno dei quattro backend packages che lo rendono disponibile: GLMakie.jl, CairoMakie.jl, WGLMakie.jl, RPRMakie.

4.3 PyPlot

Il package PyPlot è l'interfaccia di Julia per poter utilizzare il sottomodulo `pypplot` del modulo `matplotlib` dell'ecosistema Python.

Matplotlib penso sia, in assoluto, il più completo software disponibile per la grafica dei dati.

E' documentato da un file PDF di quasi 5.000 pagine, non aggiornatissimo, che troviamo all'indirizzo <https://matplotlib.org/2.0.2/Matplotlib.pdf>.

All'indirizzo <https://matplotlib.org/3.5.1/users/index.html> troviamo la stessa documentazione, in linea, aggiornata al momento in cui scrivo (maggio 2022).

Se abbiamo installato il package PyCall possiamo accedere a tutte le funzioni di `matplotlib`.

Installando il package PyPlot possiamo accedere alle funzioni del sottomodulo `pypplot` che qui interessa.

All'indirizzo <https://buildmedia.readthedocs.org/media/pdf/pyplotjl/latest/pyplotjl.pdf> troviamo un manuale di PyPlot che non è gran che ma è sempre meglio della documentazione ermetica che troviamo su GitHub.

Spero che l'uno e l'altra diventino più comprensibili dopo aver letto questo mio tentativo di divulgazione per principianti.

Dal momento che mi rivolgo a principianti limito anche gli argomenti alle tre cose più ricorrenti nella grafica dei dati: corrispondenze sul piano cartesiano, grafici a barre (detti anche a canne d'organo) e grafici a torta.

Per fare queste cose abbiamo tre funzioni, rispettivamente `plot()`, `bar()` e `pie()`.

4.3.1 `plot()`

La funzione `plot()` funziona esattamente come l'omonima funzione che abbiamo visto parlando del package `Plots`.

Unica differenza, gli array denominati `x` e `y` da cui partiamo devono essere dei vettori, cioè con elementi tra parentesi quadre, separati da virgola o punto e virgola. La funzione

`plot(x, y)`

ne evidenzia la corrispondenza su un piano cartesiano, con il vettore `x` sulle ascisse e il vettore `y` sulle ordinate.

Per default la corrispondenza viene evidenziata con una linea continua di colore blu.

Possiamo ottenere altri colori inserendo tra le parentesi tonde, dopo i primi due argomenti e separando con una virgola, il parametro opzionale

"<iniziale_colore>"

dove <iniziale_colore> è la lettera iniziale del nome del colore in lingua inglese (y yellow, m magenta, c cyan, r red, g green, b blue, w white) con la sola eccezione del nero che viene indicato con `k` per non confonderlo con il blu.

Con lo stesso parametro opzionale possiamo ottenere altri modi di evidenziare le corrispondenze tra i dati utilizzando le seguenti sigle: - per la linea continua, : per la linea punteggiata, -- per una linea a trattini, + per il relativo simbolo, o per un pallocco, * per un asterisco, . per un punto, x per il relativo simbolo a crocetta, s per un quadratino, d per il quadratino a diamante, ^ per un triangolino, v per un triangolino rovesciato, > per un triangolino puntato a destra, < per un triangolino puntato a sinistra.

Nel costruire il grafico abbiamo a disposizione le funzioni

`title("<stringa>")`

`xlabel("<stringa>")`

`ylabel("<stringa>")`

`grid()`

rispettivamente per dare un titolo al grafico, per descrivere l'asse delle ascisse, per descrivere l'asse delle ordinate e per disegnare una griglia.

Possiamo inserire le istruzioni nella REPL e vedremo progressivamente l'effetto che fanno.

Per salvare il grafico che abbiamo costruito abbiamo a disposizione la funzione

`savefig("<stringa_percorso_nome_grafico>")`

con disponibili i formati corrispondenti alle estensioni `.png`, `.jpg`, `.svg` e `.pdf`.

Possiamo altresì radunare le istruzioni in uno script che salviamo con estensione `.jl` per poterlo eseguire quando vogliamo con

`julia <nome_file>.jl`

Nello script possiamo inserire l'istruzione

`show()`

per vedere il grafico.

Se vogliamo utilizzare questa istruzione dobbiamo inserirla come ultima dello script, comunque dopo quella per salvare il grafico.

La funzione `plot()` ci serve anche per tracciare i grafici di funzione.

In questo caso il vettore delle `x` lo costruiamo con la funzione `range()` che ha la seguente sintassi

`range(<valore_iniziale>, stop = <valore_finale>, length = <elementi>)`

dove

<valore_iniziale> indica il punto da cui comincia il grafico,

<valore_finale> indica il punto dove finisce il grafico,

<elementi> indica il numero dei valori equidistanti contenuti tra il valore iniziale e il valore finale.

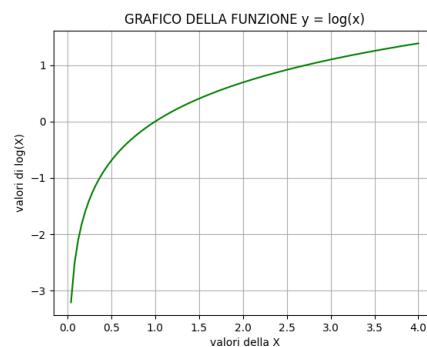
Il vettore per le ordinate possiamo costruirlo scrivendo la funzione con la dot syntax. Cosa molto pericolosa in quanto basta mettere un punto nel posto sbagliato per creare un disastro.

Qui l'unico modo di schivare il pericolo è quello di costruire il vettore delle ordinate ricorrendo alla funzione `map()`.

Qualche esempio per capirci.

```
using PyPlot
title("GRAFICO DELLA FUNZIONE  $y = \log(x)$ ")
xlabel("valori della X")
ylabel("valori di  $\log(X)$ ")
grid()
x = range(0, stop=4, length=100)
y = map(x->log(x), x)
plot(x, y, "g")
savefig("/home/vittorio/Immagini/logaritmo.png")
show()
```

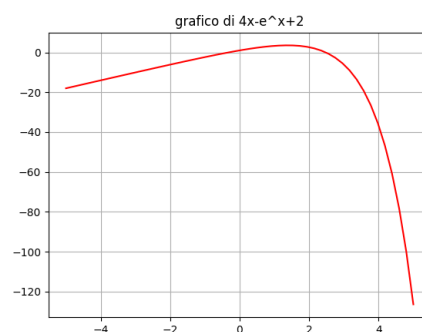
Con lo script ci viene mostrato e viene salvato all'indirizzo indicato il seguente grafico di funzione



Con quest'altro script

```
using PyPlot
title("grafico di  $4x - e^x + 2$ ")
grid()
x = range(-5, stop=5, length=50)
y = map(x->4*x-exp(1)^x+2, x)
plot(x,y,"r")
savefig("/home/vittorio/Immagini/esponenziale.png")
show()
```

ci viene mostrato e viene salvato all'indirizzo indicato il seguente grafico di funzione



4.3.2 bar()

La funzione `bar()` è deputata alla costruzione di grafici a barre, detti anche a canne d'organo.

I suoi due principali argomenti sono:

- . una tupla di stringhe per gli identificatori delle barre,
- . un vettore di valori per l'altezza delle barre.

Il primo ad indicare a cosa si riferisce il dato evidenziato da ciascuna barra, il secondo ed evidenziare il valore di quel dato.

I due principali parametri opzionali sono:

- . la larghezza della barra, indicata con la sintassi
`width = <valore>`
dove `<valore>` è compreso tra 0 e 1 (con il valore 1 le barre sono unite tra loro)
- . un vettore di stringhe per il colore delle barre indicato con la sintassi
`color = <vettore>`
dove le stringhe per i colori sono le stesse viste per la funzione `plot()`.

Questo è ciò che basta per creare i nostri grafici a barre da principianti.

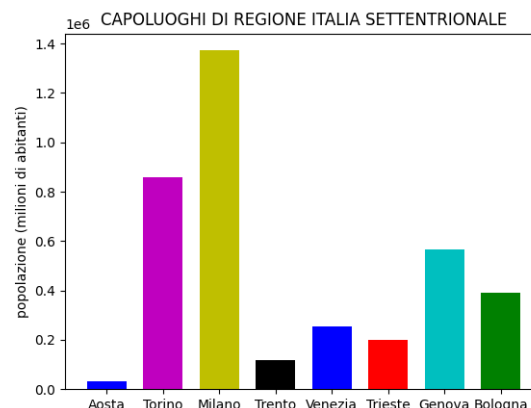
Se vogliamo rendere visibili le differenze tra la popolazione dei capoluoghi delle regioni del Nord Italia che abbiamo nella seguente tabella

Capoluogo	Abitanti
Aosta	33523
Torino	858205
Milano	1374582
Trento	118879
Venezia	256083
Trieste	200609
Genova	566410
Bologna	391686

possiamo farlo con il seguente script

```
using PyPlot
l = ("Aosta", "Torino", "Milano", "Trento", "Venezia", "Trieste", "Genova", "Bologna")
x = [33523, 858205, 1372582, 118879, 256083, 200609, 566410, 391686]
c = ["b", "m", "y", "k", "b", "r", "c", "g"]
ylabel("popolazione (milioni di abitanti)")
title("CAPOLUOGHI DI REGIONE ITALIA SETTENTRIONALE")
bar(l, x, width=0.7, color = c)
savefig("/home/vittorio/Immagini/popolazione.png")
show()
```

che visualizza e salva il seguente grafico



Ovviamente occorre badare a che gli elementi di tuple e array siano ordinati come si deve.

4.3.3 pie()

La funzione `pie()` è deputata alla costruzione di grafici a torta.

L'argomento base da passare alla funzione è:

- . un vettore contenente i valori delle fette.

Il principale argomento opzionale è:

- . un vettore di stringhe per le etichette delle fette da passare con la sintassi

`labels = <vettore>`

In mancanza di indicazioni al riguardo le fette vengono differenziate con colori scelti a caso.

Se vogliamo scegliere i colori delle fette abbiamo l'argomento opzionale:

- . vettore di stringhe per il colore delle fette indicato con la sintassi

`colors = <vettore>`

dove le stringhe per i colori sono le stesse viste per la funzione `plot()`.

Se vogliamo rendere visibile la ripartizione della popolazione mondiale sui continenti che abbiamo nella seguente tabella

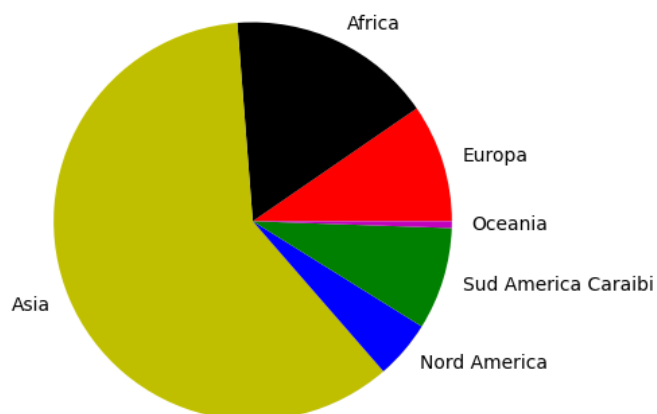
Continente	Popolazione (milioni)
Europa	750
Africa	1.300
Asia	4.700
Nord America	370
Sud America e Caraibi	650
Oceania	43

possiamo farlo con il seguente script

```
using PyPlot
x = [750,1300,4700,370,650,43]
l = ["Europa","Africa","Asia","Nord America", "Sud America Caraibi","Oceania"]
c = ["r","k","y","b","g","m"]
title("RIPARTIZIONE DELLA POPOLAZIONE SUI CONTINENTI")
pie(x, labels = l, colors = c)
savefig("/home/vittorio/Immagini/continenti.png")
show()
```

che visualizza e salva il seguente grafico

RIPARTIZIONE DELLA POPOLAZIONE SUI CONTINENTI



5 Julia per la scienza e il deep learning

Quanto visto finora ci mette in grado di fare, pur da dilettanti, cose di un certo interesse con il linguaggio Julia.

Grazie alla ricchezza delle librerie che si installano con il software di base, come abbiamo visto nel Paragrafo 2.11, siamo anche in grado, senza installazioni aggiuntive, di utilizzare funzioni di algebra lineare e di statistica di base.

Abbiamo anche visto, nel Paragrafo 2.14, come possiamo ricorrere, per cose più impegnative, a moduli del linguaggio Python attraverso il pacchetto PyCall.

In proposito devo dire che, nel momento in cui scrivo, questo pacchetto è stato affiancato da una nuova versione, che si chiama PythonCall, che non ho trattato in quanto complica molto la vita, richiedendo una reinstallazione dei moduli Python da utilizzare. Speriamo che il vecchio PyCall non venga deprecato e messo fuori servizio.

Il ricorso a moduli Python, comunque, è una buona opportunità ma non può certo essere la via maestra per fare seriamente, con il linguaggio Julia, tutto ciò che è richiesto per ricerche scientifiche, data science e deep learning: tanto varrebbe allora utilizzare il linguaggio Python. Anche perché quando si utilizzano moduli Python con Julia, il pregio della velocità computazionale di Julia sparisce.

E' per questo che la comunità di Julia ha elaborato ed elabora continuamente pacchetti propri. Quelli che troviamo nel Repository di Julia, di cui ho parlato nel Paragrafo 1.2.

5.1 Calcolo simbolico con Symbolics.jl

Per il calcolo simbolico possiamo ricorrere ai comandi del modulo Python SymPy attraverso il package Julia PyCall o addirittura utilizzare l'interfacciamento diretto attraverso il package Julia SymPy. Lo abbiamo visto in esempi nel Paragrafo 2.15.

In alternativa abbiamo a disposizione il pacchetto Symbolics.jl, programmato in linguaggio Julia e facente parte direttamente dell'ecosistema Julia.

Propongo un confronto.

Sia da calcolare la derivata della funzione $y = \sin x^2 \exp x$.

Se utilizziamo il modulo Python SymPy procediamo in questo modo

```
julia> using SymPy
julia> x = symbols("x");
julia> derivata = diff(sin(x^2)*exp(x), x);
julia> print(simplify(derivata))
(2*x*cos(x^2) + sin(x^2))*exp(x)
```

Se utilizziamo il package Julia Symbolics procediamo in questo modo

```
julia> using Symbolics
julia> @variables x;
julia> derivata = Symbolics.derivative(sin(x^2)*exp(x), x);
julia> print(simplify(derivata))
sin(x^2)*exp(x) + 2x*exp(x)*cos(x^2)
```

Stessa quantità di istruzioni, ovviamente diverse tra loro, per arrivare allo stesso risultato, meglio espresso nel caso di SymPy. Forse per un errore di gioventù la funzione `simplify` di Symbolics non raccoglie il termine $\exp(x)$.

Circa la maggiore velocità di elaborazione ottenibile con l'uso di Symbolics questo piccolo esempio non può ovviamente dimostrare nulla.

Da dilettanti, tutto sommato, direi che, una volta che abbiamo imparato ad usare SymPy, potremmo avvalercene sia programmando in Python sia programmando in Julia.

A chi voglia cimentarsi con Symbolics dedico questo paragrafo, che ne guida un utilizzo di base, esonerando così il dilettante, in mancanza di qualsiasi abbordabile indicazione in lingua italiana e facilmente comprensibile, dal dover affrontare la consultazione della documentazione di Julia, notoriamente carente sul piano della chiarezza.

Con la procedura descritta nel Paragrafo 2.14 installiamo il pacchetto Symbolics e, per utilizzarlo, scriviamo nella REPL o come prima riga del nostro script il comando `using Symbolics`.

Può essere utile installare un altro pacchetto di arricchimento, chiamato Latexify, che consente di tramutare qualsiasi espressione prodotta da Julia in linguaggio $\text{\LaTeX}$, in modo da poter facilmente inserire in documenti prodotti con $\text{\LaTeX}$, con copia e incolla, quando necessario per ottenere la stessa espressione scritta nell'insuperabile ben noto stile $\text{\LaTeX}$. Per utilizzare anche questo pacchetto aggiungiamo il comando `using Latexify`.

5.1.1 Operazioni di base

Dichiarazione dei simboli

I simboli non numerici che intendiamo usare nelle nostre elaborazioni vanno preventivamente dichiarati.

La dichiarazione avviene usando la macro `variables` con la sintassi

```
@variables <simbolo>, <simbolo>, ...
```

dove `<simbolo>` è una lettera dell'alfabeto latino direttamente prodotta con la tastiera o una lettera dell'alfabeto greco prodotta con le dizioni `alpha`, `beta`, `gamma`, `delta`, `epsilon`, `zeta`, `eta`, `theta`, `iota`, `kappa`, `lamda`, `mu`, `nu`, `xi`, `omicron`, `pi`, `rho`, `sigma`, `tau`, `upsilon`, `phi`, `chi`, `psi`, `omega` precedute da `\` e con pressione del tasto TAB dopo averle scritte.

Nello stesso modo dovremo scrivere questi simboli espressi con lettere dell'alfabeto greco quando li inseriremo nelle espressioni.

Il fatto che la macro si chiami `variables` non ha niente a che vedere con le variabili del linguaggio Julia.

Costruzione delle espressioni

Le espressioni matematiche si compongono utilizzando i simboli dichiarati, funzioni riconosciute da Symbolics e numeri, combinati con gli operatori aritmetici del linguaggio Julia (`^` `/` `*` `/` `+` `-`).

L'espressione può essere assegnata a una variabile.

Esempio:

```
e = sin(x)^2 + cos(x)^2 - x + 2*x*y
```

assegna alla variabile `e` l'espressione matematica $\sin^2(x) + \cos^2(x) - x + 2xy$

Possiamo creare un'espressione da una stringa utilizzando la sintassi:

```
<nome_espressione> = eval(Meta.parse("<stringa>"))
```

```
<nome_espressione> = eval(Meta.parse(<nome_stringa>))
```

La stringa può essere un nome o una frase qualsiasi ma, ovviamente, a noi servirà che sia un'espressione matematica.

Nel primo caso scriviamo la stringa tra le parentesi e tra apici.

Nel secondo caso importiamo una variabile stringa già esistente con un suo nome.

In questo modo possiamo acquisire come espressione matematica elaborabile in Symbolics un'espressione generata al di fuori di Symbolics, per esempio introdotta dall'utente da tastiera come risposta all'invito di input da uno script Julia.

Esempi:

```
e = eval(Meta.parse("4*x^2-5"))
```

assegna alla variabile `e` l'espressione matematica $4x^2 - 5$.

```
e = eval(Meta.parse(formula))
```

assegna alla variabile `e` l'espressione contenuta in una stringa denominata `formula`.

Sostituzione di simboli

Le espressioni matematiche di Symbolics sono immutabili.

E' tuttavia possibile generare altre espressioni matematiche sostituendo simboli di una espressione matematica con altri simboli o valori numerici.

A questo scopo dobbiamo costruire un dizionario, che possiamo chiamare *sostituzioni*, indicando in chiave il o i simboli da sostituire e in loro corrispondenza l'elemento o gli elementi da mettere in sostituzione (altri simboli o valori numerici).

Poi possiamo effettuare le sostituzioni con la sintassi
`substitute(<vecchia_espressione>, sostituzioni)`

Esempio:

Data l'espressione *e* dell'ultimo esempio del paragrafo precedente, dichiarate le variabili *x* e *y*, con

```
sostituzioni = Dict([x=>y]) e  
substitute(e, sostituzioni)  
generiamo l'espressione  $4y^2 - 5$ .
```

Con

```
sostituzioni = Dict([x=>2]) e  
substitute(e, sostituzioni)  
generiamo l'espressione matematica  $4 * 2^2 - 5$ , cioè il numero 11.
```

In ogni caso l'espressione *e* rimane $4x^2 - 5$.

Se vogliamo salvare l'espressione modificata dobbiamo assegnare il risultato della sostituzione ad una nuova variabile.

Ad esempio con

```
new_e = substitute(e, sostituzioni)  
creeremo una variabile, chiamata new_e e che, nel primo caso, conterrebbe l'espressione  $4y^2 - 5$   
e, nel secondo caso, conterrebbe il valore 11.
```

Dall'espressione numerica al numero

Quando un sistema di calcolo simbolico compie calcoli su numeri fornisce il risultato in una espressione numerica.

Nei sistemi di calcolo simbolico puro, come quello del modulo SymPy di Python, se questo risultato è un numero che esiste, l'espressione numerica coincide con il numero stesso e non ci accorgiamo nemmeno che è un'espressione numerica.

Se il risultato è un numero che non esiste e che non si può calcolare, come avviene per i numeri irrazionali, l'espressione numerica è, in realtà, un simbolo.

La funzione `sqrt(8)` del SymPy di Python fornisce il risultato $2\sqrt{2}$.

L'oggetto che contiene questo risultato ha una funzione membro, che si chiama `evalf()`, che forza la traduzione dell'espressione numerica simbolica in numero approssimato.

Nel linguaggio Julia il sistema di calcolo simbolico non è puro ma ibrido e qualsiasi espressione numerica viene forzata a diventare numero, magari approssimato.

Pertanto se in Julia utilizziamo i comandi del modulo SymPy di Python utilizzando il package Pycall abbiamo un sistema di calcolo simbolico puro.

Se utilizziamo i package SymPy o Symbolics di Julia abbiamo un sistema di calcolo simbolico ibrido.

Nel primo caso, il comando `sqrt(8)` fornisce il risultato $2\sqrt{2}$.

Nel secondo caso, il comando `sqrt(8)` fornisce il risultato 2.8284271247461903.

Visualizzazione

Per visualizzare il contenuto di variabili e i risultati che via via otteniamo dalle nostre elaborazioni, dal momento che lavoriamo in Julia, abbiamo ovviamente a disposizione la funzione `print()`, con la quale, tuttavia, otteniamo la visualizzazione delle espressioni così come si inseriscono da tastiera e non vi è modo di migliorare l'output in tal senso.

Se ci serve evidenziare i risultati delle nostre elaborazioni in maniera tipograficamente eccellente possiamo ricorrere al package Latexify ed al suo comando `latexify`.

Per esempio, se abbiamo nella variabile *derivata* il risultato ottenuto nell'esempio fatto in premessa:

```
sin(x^2)*exp(x) + 2x*exp(x)*cos(x^2)
```

con i comandi

```
using Latexify
```

```
latexify(derivata)
```

otteniamo a terminale il codice Latex per inserire in un documento Latex la nostra derivata

```
julia> latexify(derivata)
L"\begin{equation}
\sin\left( x^2 \right) e^x + 2 x e^x \cos\left( x^2 \right)
\end{equation}"
```

Per esempio, inserendo con copia e incolla, tra due simboli \$ e \$ la riga centrale tra `\begin` (equation) e `\end` nel documento che sto scrivendo con l'editor LyX, dopo aver scelto lo strumento INSERISCI MATEMATICA ottengo

$$\sin(x^2)e^x + 2xe^x \cos(x^2)$$

che è un po' meglio da vedere e da leggere.

5.1.2 Elaborazione delle espressioni

Semplificazioni

Quando creiamo un'espressione dalla tastiera o importando una stringa, Symbolics la acquisisce apportando già le più immediate semplificazioni.

Per esempio, se inseriamo nella variabile `e` l'espressione

$$\sin^2(x) + \cos^2(x) - x + 2xy + 4x - \frac{xy}{2}$$

digitando la seguente istruzione

```
e = sin(x)^2 + cos(x)^2 - x + 2*x*y + 4*x - (x*y)/2
```

l'espressione che viene creata da Symbolics è

$$3x + (3/2)*x*y + \sin(x)^2 + \cos(x)^2$$

meglio «latexificabile» così

$$3x + \frac{3}{2}xy + \sin^2(x) + \cos^2(x)$$

nella quale abbiamo già $3x$ al posto di $-x + 4x$ e $\frac{3}{2}xy$ al posto di $2xy - \frac{xy}{2}$

Ma la nostra espressione contiene un'altra possibile semplificazione, meno immediata: la somma dei quadrati di $\sin(x)$ e $\cos(x)$, infatti, è uguale a 1, qualunque sia il valore di x .

Per eseguire le semplificazioni meno immediate Symbolics ci offre la funzione `simplify()` il cui argomento può essere un'espressione creata al momento o il nome di un'espressione esistente.

Esempi:

```
simplify(sin(x)^2 + cos(x)^2)
```

fornisce il risultato 1;

```
simplify(e)
```

 con riferimento all'espressione `e` del precedente esempio

fornisce il risultato

$$1 + 3x + (3/2)*x*y$$

meglio «latexificabile» così

$$1 + 3x + \frac{3}{2}xy.$$

Espansioni polinomiali

Per espandere un'espressione polinomiale abbiamo la funzione `expand()`, il cui argomento è un'espressione inserita direttamente o richiamata.

```
expand((a+b)^3)
```

 produce l'espressione $a^3 + 3a^2b + 3ab^2 + b^3$

Se abbiamo l'espressione `e = (x - 3)*(x + 1)`,

```
expand(e)
```

 produce l'espressione $-3 - 2x + x^2$

```
expand((a+b)*(a-b))
```

 produce l'espressione $a^2 - b^2$

Operazioni sui polinomi

Per sommare, sottrarre, moltiplicare ed elevare a potenza possiamo usare i soliti operatori `+` `-` `*` e `^` di Julia.

Esempi:

Siano A e B i due polinomi

$$A = x^4 - 2x^3 - 8x + 16$$

$$B = 2x^4 + 8x$$

$$A + B \text{ ritorna } 3x^4 - 2x^3 + 16$$

$$A - B \text{ ritorna } -x^4 - 2x^3 - 16x + 16$$

$$A * B, \text{ associando il risultato a } \text{expand}(), \text{ ritorna } 2x^8 - 4x^7 - 8x^5 + 16x^4 - 64x^2 + 128x$$

$$B ** 2, \text{ associando il risultato a } \text{expand}(), \text{ ritorna } 4x^8 + 32x^5 + 64x^2$$

Per la divisione Symbolics non è attrezzato e occorrerebbe ricorrere al package aggiuntivo `MultivariatePolynomials.jl`.

5.1.3 Calcolo

Symbolics possiede tutte le funzioni matematiche preconfezionate che ritroviamo nel linguaggio Julia e le possiamo applicare anche a simboli semplicemente richiamandole o richiamandole con la sintassi `Symbolics.<nome_funzione>`.

Sempre nell'ambito del calcolo, con la funzione

`Symbolics.derivative(<funzione_da_derivare>, <variabile_di_derivazione>)`

calcoliamo la derivata.

Con

```
julia> using Symbolics
julia> @variables x;
julia> Symbolics.derivative(log(x), x)
otteniamo
1 / x
```

Per altri tipi di calcolo, come integrali e limiti, Symbolics non è direttamente attrezzato ed occorrerebbe installare i pacchetti aggiuntivi `SymbolicNumericIntegration.jl` e `SymbolicUtils.jl`.

5.1.4 Risolutori

Occorre installare il pacchetto aggiuntivo `Nemo.jl` e, per coprire tutte le possibili tipologie di equazioni, anche il pacchetto `Roots.jl`.

5.1.5 Matrici

Per costruire una matrice lavorando con Symbolics usiamo la stessa sintassi che usiamo lavorando in Julia.

In Julia con

```
[1 2; 4 2]
```

costruiamo la matrice

$$\begin{vmatrix} 1 & 2 \\ 4 & 2 \end{vmatrix}$$

In Symbolics con

```
@variables a,b,c,d
```

```
[a b; c d]
```

costruiamo la matrice

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix}$$

Lo stesso dicasi per i vettori.

```
[a b]
```

costruisce il vettore riga $\begin{vmatrix} a & b \end{vmatrix}$

[a; b]

costruisce il vettore colonna $\begin{bmatrix} a \\ b \end{bmatrix}$

Con gli operatori +, -, *, / e ^ possiamo fare la somma, la differenza, la moltiplicazione, la divisione e l'elevamento a potenza di matrici.

Se dichiariamo anche l'uso del package LinearAlgebra, che fa parte della dotazione standard di Julia, possiamo calcolare la matrice inversa con

inv(<matrice>)

e il determinante con

det(<matrice>).

Esempi:

Previo

```
using Symbolics
```

```
@variables x
```

Siano $A = \begin{bmatrix} x & 2 \\ 1 & 3 \end{bmatrix}$ e $B = \begin{bmatrix} 3 & 2 \\ 4 & 1 \end{bmatrix}$

$$A + B = \begin{bmatrix} 3+x & 4 \\ 5 & 4 \end{bmatrix}$$

$$A * 3 = \begin{bmatrix} 3x & 6 \\ 3 & 9 \end{bmatrix}$$

$$A * B = \begin{bmatrix} 8+3x & 2+2x \\ 15 & 5 \end{bmatrix}$$

$$A ^ 2 = \begin{bmatrix} 2+x^2 & 6+2x \\ 3+x & 11 \end{bmatrix}$$

$$A ^ -1 = \begin{bmatrix} \frac{3}{-2+3x} & \frac{-2}{-2+3x} \\ \frac{-1}{-2+3x} & \frac{x}{-2+3x} \end{bmatrix}$$

$$B ^ -1 = \begin{bmatrix} -0.2 & 0.4 \\ 0.8 & -0.6 \end{bmatrix}$$

A.det() = -2 + 3x

B.det() = -5

5.2 Statistica inferenziale

Gli strumenti che ci offre Julia sono i pacchetti:

Distributions.jl, per gestire e campionare da distribuzioni di probabilità,

HypothesisTests.jl, per eseguire test d'ipotesi parametrici e non parametrici,

GLM.jl, per modelli lineari, regressioni e ANOVA.

5.3 Matematica superiore

Posso citare i pacchetti:

DifferentialEquations.jl, per risolvere equazioni differenziali di ogni tipo,

JuMP.jl, per l'ottimizzazione lineare, intera, quadratica e non lineare,

Optim.jl, per l'ottimizzazione univariata e multivariata senza vincoli o con vincoli generici.

5.4 Machine learning

MLJ.jl, è il framework principale per il Machine Learning in Julia e svolge lo stesso ruolo di scikit-learn in Python, offrendo un'interfaccia unica per connettere e confrontare decine di modelli diversi (alberi di decisione, clustering, macchine a vettori di supporto).

5.5 Manipolazione dati

DataFrames.jl è lo strumento fondamentale per fare analisi ed elaborazione dati nel linguaggio Julia e svolge lo stesso ruolo di Pandas in Python.

5.6 Deep learning

Flux.jl, molto simile a PyTorch, per il deep learning classico.

* * *

Per questi ultimi campi di applicazione, sicuramente non alla portata dei dilettanti cui dedico i miei manuali, non mi dilungo in particolari e invito il lettore, se interessato a approfondimenti, ad avviare la ricerca di documentazione da <https://juliapackages.com/>.

Buone istruzioni introduttive si possono anche trovare con strumenti di IA come Gemini, Copilot o ChatGPT.